

Opus Publica

constitution.yaml Schema Reference

The Machine-Readable Constitution (MRC) — Field, Schema, and Validation Reference

Version 1.0

Issued 09 August 2026

Document Type	Implementation Specification (RC2 Machine-Readable Constitution)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineers authoring or modifying constitutional provisions; tooling engineers building generators, validators, and policy engines
Classification	Normative reference for RC2 constitution/ repository
Effective Date	09 August 2026
Supersedes	RC1 prose-only Constitution provisions (machine-readable form is new)
Review Cadence	On every schema change; before every RC milestone transition
Document ID	OP-SCHEMA-REF-001

This reference is the normative specification of the machine-readable form of the Opus Publica Engineering Constitution. Every field, every schema, and every validation rule documented here is binding on the constitution/ Git repository. Where this reference and a generated artifact disagree, this reference prevails.

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Chapter 1 — The Constitution as Data

1.1 The Constitution as Data

For three years the Opus Publica Engineering Constitution has existed as prose. Three hundred and three pages of Volume I; two hundred and sixty-nine pages of Volume II; one hundred and ninety-six pages of Playbook; one hundred and ninety-one pages of Traceability Matrix; three hundred and twenty-six pages of Certification Checklist. Together, well over a thousand pages, written in careful natural-language English, cross-referenced by section number, governed by RFC 2119 vocabulary, audited quarterly by the Release Certification Authority. The Constitution has been the supreme engineering authority for the platform. It has also been, until now, fundamentally a document that humans read and humans enforce.

The RC2 Evolution Roadmap proposed a single architectural change with outsized consequences: convert the Constitution from a document into a database. The Machine-Readable Constitution (MRC) is a Git repository named `constitution/`, in which every provision — every invariant, every rule, every contract, every canonical field, every event, every SLO, every threshold, every state, every transition, every certification criterion — is encoded as a single YAML file conforming to a JSON Schema 2020-12 definition. The prose Constitution becomes a generated artifact, produced from the MRC by deterministic document generators; the traceability matrix becomes a derived view, queried from the MRC by graph traversal; the certification checklist becomes a compiled form, projected from the MRC's certification criteria. The MRC is no longer the Constitution's reflection; the MRC IS the Constitution.

The Prime Directive of the Machine-Readable Constitution

In RC2 and beyond, the `constitution/` Git repository IS the Engineering Constitution. Every derived artifact — the prose Constitution, the Playbook, the Traceability Matrix, the Certification Checklist, the OSCAL Assessment Results, the OPA policy bundle, the Backstage Software Catalog entry — is generated from the MRC. Where a generated artifact and the MRC disagree, the MRC prevails and the disagreement is filed as a bug against the generator. There are no manual edits to derived artifacts.

This reference is the normative specification of the MRC. It documents every field of every provision class, every sub-schema, every validation rule, and every authoring workflow. It is written for two audiences. The first is the engineer who authors or amends a provision: this reference tells them which fields are required, which values are valid, and which cross-field constraints their YAML must satisfy. The second is the tooling engineer who builds a generator, a validator, a policy engine, or a dashboard: this reference tells them the schema they must consume, the validation rules they must enforce, and the version-compatibility guarantees they must preserve. Both audiences need the same reference because both audiences operate on the same data.

1.2 Purpose, Audience, Prerequisites

The purpose of this reference is to specify, completely and unambiguously, the machine-readable form of the Engineering Constitution. ‘Completely’ means that every field of every provision class is documented with its type, its requiredness, its valid values, an example, and the validation rules that constrain it. ‘Unambiguously’ means that a tooling engineer reading this reference can implement a conforming validator without consulting any other source, and a provision author reading this reference can write a conforming YAML file without consulting any other source.

The intended audiences are:

Audience	Primary Use of This Reference
Provision Authors	Field reference when writing or amending a YAML provision file. Chapter 3 (universal schema) and Chapter 4 (class-specific schemas) are the primary chapters; Chapter 8 (authoring workflow) is the procedural companion.
Tooling Engineers	Schema reference when building or maintaining a generator, validator, policy engine, dashboard, or graph builder. Chapter 3, Chapter 5 (cross-cutting sub-schemas), Chapter 6 (validation rules), and Appendix A (consolidated JSON Schema) are the primary chapters.
Release Certification Authority	Reference for understanding what a CC-* criterion encodes and how the MRC maps to the binary PASS/FAIL regime. Chapter 4 §4.10 and Chapter 7 (worked examples) are the primary chapters.
Architects and ADR Authors	Reference for understanding how an ADR's @requires annotations resolve to MRC provisions and how the traceability graph edges are populated. Chapter 5 §5.1 (traceability object) is the primary chapter.
Auditors and External Reviewers	Normative reference for verifying that the MRC conforms to its own schema and that derived artifacts are reproducible from the MRC. The whole document is the reference; Appendix A and Appendix C are the operational artifacts.

This reference assumes familiarity with the following:

- **YAML.** YAML 1.2.2 (the current authoritative specification, yaml.org/spec/1.2.2/), including block scalars (| and >), flow collections ([] and { }), and quoting rules.

- **JSON Schema.** JSON Schema 2020-12 (json-schema.org/draft/2020-12/schema), including the \$schema, \$id, type, properties, required, enum, pattern, allof, oneof, if/then/else, and \$ref keywords.
- **SemVer.** Semantic Versioning 2.0.0 (semver.org), including the MAJOR.MINOR.PATCH structure and the precedence rules.
- **RFC 2119.** RFC 2119 (rfc-editor.org/info/rfc2119) requirement vocabulary: MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, OPTIONAL.
- **The Constitution.** The Opus Publica Engineering Constitution, at least at the level of its chapter structure (Vol I Ch 1–25, Vol II Ch 26–36) and its provision-class taxonomy (invariants, rules, contracts, fields, events, SLOs, thresholds, states, transitions, criteria).
- **CUE (recommended).** CUE (cuelang.org), at least at the level of the package, import, field, list, disjunction (|), and if-then syntax. CUE is the authoring language for the parts of the MRC where constraints are non-trivial; this reference documents the CUE source in Chapter 3 §3.4.

1.3 Scope

This reference specifies every field of every provision class in the MRC. ‘Every field’ means: every field of the universal provision schema (Chapter 3, eighteen fields); every class-specific field of every provision class (Chapter 4, ten classes); every sub-field of every cross-cutting sub-schema (Chapter 5, four sub-schemas). ‘Every provision class’ means the ten classes enumerated in Chapter 4: invariant, rule, contract, field, event, slo, threshold, state, transition, criterion.

This reference does NOT specify:

- **Provision content.** The substantive content of any individual provision. The MRC currently encodes 38 invariants, 47 rules, 20 contracts, 276 fields, 34 events, 10 SLOs, 103 thresholds, 13 states, 17 transitions, and 600+ certification criteria. This reference documents the schema those provisions conform to; the provisions themselves are the MRC's content, documented in the generated Constitution volumes.
- **Tooling implementation.** The implementation of the OPA/Rego policy engine, the Neo4j traceability graph, or the document generators. Those are operational concerns documented in the RC2 Evolution Roadmap and the Engineering Playbook.
- **Governance process.** The governance process for amending the Constitution. The process is documented in the Engineering Constitution Vol I Chapter 24 (Amendment Process) and Vol II Chapter 36 (Meta-Constitution). This reference documents only the mechanical workflow (§8) for changing a YAML file.

1.4 Document Conventions

This reference uses the following typographic and naming conventions:

Convention	Meaning
------------	---------

monospace	Field names, YAML keys, file paths, command lines, and code fragments appear in monospace (Consolas 9.5 pt). Example: the <code>source_chapter</code> field.
“quotation marks”	Literal string values, including enum members and example values, appear in curly double quotation marks. Example: the value “SEV-1”.
RFC 2119 keywords	MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL appear in small capitals and carry the precise meanings defined in RFC 2119. A field marked REQUIRED MUST be present; a field marked OPTIONAL MAY be omitted.
Field naming	YAML keys use snake_case throughout: <code>source_chapter</code> , <code>required_evidence</code> , <code>runtime_monitor</code> . JSON Schema property names match the YAML keys exactly. CUE field names match the YAML keys exactly.
Version conventions	Document version (this reference is Version 1.0) follows MAJOR.MINOR form. Schema version (the \$id URI) follows MAJOR.MINOR.PATCH form per JSON Schema 2020-12. Provision version (the version field of each provision) follows SemVer 2.0.0.
Citations	External standards are cited by their canonical short name (JSON Schema 2020-12, YAML 1.2.2, RFC 2119, SemVer 2.0.0) and resolved in Appendix D. Internal cross-references use the form “§N.M” or “Chapter N”.
Code blocks	YAML, JSON Schema, CUE, and shell command examples appear in shaded code blocks. Every code block is syntactically valid; examples that span a logical unit carry a leading comment identifying the source file path.

1.5 References

The following normative and informative references are cited throughout this reference. The full bibliography, with URLs and access dates, is in Appendix D.

Reference	Role	Source
JSON Schema 2020-12	normative	json-schema.org/draft/2020-12/schema

YAML 1.2.2	normative	yaml.org/spec/1.2.2/
CUE	normative (for §3.4)	cuelang.org
OpenAPI 3.1	informative (for §9.4)	spec.openapis.org/oas/v3.1.0
RFC 2119	normative	rfc-editor.org/info/rfc2119
SemVer 2.0.0	normative	semver.org
Conventional Commits 1.0.0	normative (for §8)	conventionalcommits.org/en/v1.0.0/
Opus Publica RC1 Engineering Constitution	normative	Vol I Ch 1–25, Vol II Ch 26–36 (this repository)
Opus Publica RC2 Evolution Roadmap	informative	Opus_Publica_RC2_Evolution_Roadmap_v1.0, Chapter 2

Chapter 2 — The Repository Structure

2.1 Directory Layout

The MRC is a single Git repository named `constitution/`. The repository is the authoritative source of every provision; derived artifacts (the prose Constitution, the Playbook, the Traceability Matrix, the Certification Checklist, the OSCAL Assessment Results) are generated into a sibling repository named `constitution-built/` by the document generators in CI. The full directory tree of the MRC is:

constitution/				
meta.yaml		#	repository-level metadata	
invariants/	# 38	invariants	(Vol II Ch 26)	
identity/		#	INV-I-01..07	
			INV-I-01.yaml	
			INV-I-02.yaml	
			...	
immutability/		#	INV-M-01..05	
resolvability/		#	INV-R-01..05	
completeness/		#	INV-C-01..07	
legality/		#	INV-L-01..06	
auditability/		#	INV-A-01..04	
conservation/		#	INV-V-01..04	
rules/	# 47	non-negotiable rules	(Vol I Ch 3)	
architecture/		#	ARCH-001..007	
data/		#	DATA-001..007	
publication/		#	PUB-001..008	
external/		#	EXT-001..007	
operations/		#	OPS-001..008	
process/		#	PROC-001..010	
contracts/	# 20	subsystem contracts	(Vol II Ch 27)	
			SUB-01-submission.yaml	
			SUB-02-review.yaml	
			...	
			SUB-10-doi-minter.yaml	
			...	
			SUB-20-storage.yaml	
fields/	# 276	canonical fields	(Vol II Ch 28)	
article/		#	45 fields	
author/		#	28 fields	
journal/		#	27 fields	
issue/		#	22 fields	
submission/		#	31 fields	
review/		#	24 fields	
audit/		#	18 fields	
doi/		#	16 fields	
preservation/		#	19 fields	

```

    user/                                # 21 fields
    billing/                             # 25 fields
events/                                # 34 events (Vol II Ch 29)
    EVT-01-ArticleSubmitted.yaml
    ...
    EVT-22-PublicationCompleted.yaml
    ...
slos/                                  # 10 SLOs (Vol II Ch 32)
    SLO-01-availability.yaml
    SLO-02-publication-latency.yaml
    ...
    SLO-10-crossref-deposit-latency.yaml
thresholds/                            # 103 thresholds (Vol II Ch 31)
    latency/                            # 28 thresholds
    LAT-001-api-read.yaml
    ...
    throughput/                         # 22 thresholds
    batch/                              # 19 thresholds
    queue/                              # 17 thresholds
    storage/                            # 17 thresholds
state-machine/                         # 13 states + 17 transitions (Vol I Ch 11)
    states.yaml
    transitions.yaml
    illegal-transitions.yaml
certification/                         # 600+ criteria (Vol I Ch 8 + Vol II Ch 22)
    criteria/                           # CC-* criterion files
    CC-INV-I-02.yaml
    ...
    pillars.yaml
gate.yaml                             # Production Acceptance Gate definition
schemas/                              # JSON Schema 2020-12 definitions
    provision.schema.json              # universal provision schema
    invariant.schema.json
    rule.schema.json
    contract.schema.json
    field.schema.json
    event.schema.json
    slo.schema.json
    threshold.schema.json
    state.schema.json
    transition.schema.json
    criterion.schema.json
    meta.schema.json
cue/                                   # CUE source definitions
    constitution.cue                  # generates schemas/*.schema.json
policies/                             # OPA/Rego policy modules
    INV-I-01.rego
    INV-I-02.rego

```

	PUB-001.rego
	...
generators/	# document generators (Python)
	gen_constitution_vol1.py
	gen_constitution_vol2.py
	gen_playbook.py
	gen_traceability_matrix.py
	gen_certification_checklist.py
	gen_oscal.py
	gen_opa_bundle.py
	gen_backstage_catalog.py
validators/	# validation scripts
validate_mrc.py	# structural + semantic + referential
validate_versions.py	# SemVer + amendment history
.github/workflows/	# CI pipelines
	validate-mrc.yml
	generate-derived.yml
	publish-opa-bundle.yml
README.md	

The tree is intentionally flat at the second level (one directory per provision class) so that the schema validator can map every YAML file to its schema by directory, without parsing the file first. The third level organizes provisions by topic (e.g., invariants/identity/ for INV-I-*) and is purely organizational; the validator does not enforce topic-level placement.

2.2 File Naming Conventions

Every YAML file in the MRC is named by its provision id, in kebab-case where the id contains hyphens already, and suffixed with .yaml. The naming convention is normative: the file name **MUST** equal the value of the id field within the file, in lowercase, with the conventional hyphenation. This invariant is checked by the validator and simplifies tooling: a reference to INV-I-02 in an ADR resolves to a file path by string transformation alone, without a lookup table.

ID Pattern	File Name	Example	Count
INV-{CLASS}-{NN}	INV-{CLASS}-{NN}.yaml	invariants/identity/INV-I-02.yaml	38
{DOMAIN}-{NNN}	{DOMAIN}-{NNN}.yaml	rules/architecture/ARCH-001.yaml	47
SUB-{NN}-{slug}	SUB-{NN}-{slug}.yaml	contracts/SUB-10-doi-minter.yaml	20
{entity}_{field}	{entity}_{field}.yaml	fields/article/article_doi.yaml	276
EVT-{NN}-{EventName}	EVT-{NN}-	events/EVT-22-	34

	{EventName}.yaml	PublicationCompleted.yaml	
SLO-{NN}-{slug}	SLO-{NN}-{slug}.yaml	slos/SLO-01-availability.yaml	10
{CLASS}-{NNN}-{slug}	{CLASS}-{NNN}-{slug}.yaml	thresholds/latency/LAT-001-api-read.yaml	103
(state machine)	states.yaml, transitions.yaml	state-machine/ transitions.yaml	13+17
CC-{provision-id}	CC-{provision-id}.yaml	certification/criteria/CC-INV-I-02.yaml	600+

Where a file is non-provisional (e.g., meta.yaml, pillars.yaml, gate.yaml, the schema files, the CUE source), the file name is the lowercase form of its role. These files are documented where they appear in this reference: meta.yaml in §2.3, the schemas in Chapter 3 and Appendix A, the CUE source in §3.4.

2.3 The meta.yaml File

The repository root contains a single meta.yaml file that records repository-level metadata: the schema version of the MRC itself, the effective date, the document this version supersedes, and the owning authority. The meta.yaml file is consumed by every generator and every validator as the entry point; if meta.yaml is absent or invalid, no downstream artifact is produced.

```
# constitution/meta.yaml
schema_version: 1.0.0 # SemVer of the MRC schema itself
constitution_version: 2.0.0 # SemVer of the encoded Constitution
effective_date: 2026-08-09 # ISO 8601 date this version takes effect
supersedes:
  - constitution_version: 1.9.4
    effective_date: 2026-07-15
    commit_sha: 4a2c8e91
owner: "Office of the Chief Systems Architect"
custodian: "platform-engineering@opuspublica.com"
source_documents:
  - "Opus Publica RC1 Engineering Constitution (prose)"
  - "Opus Publica RC2 Evolution Roadmap v1.0 Ch 2"
generator_pins:
  python: "3.12"
  jsonschema: "4.21"
  cue: "0.9"
  opa: "0.65"
  neo4j: "5.20"
ci_pipelines:
  - .github/workflows/validate-mrc.yml
  - .github/workflows/generate-derived.yml
```

```

- .github/workflows/publish-opa-bundle.yml
  status: active

```

The `schema_version` field is the SemVer of the MRC schema (the `provision.schema.json` and class schemas). A MAJOR bump indicates a breaking schema change (e.g., a field rename or a removed enum value); existing provisions **MUST** be migrated as part of the same commit. A MINOR bump indicates an additive change (e.g., a new optional field). A PATCH bump indicates a clarification or documentation-only change. The `constitution_version` field is the SemVer of the Constitution content itself and tracks the amendment cadence described in §8.

2.4 Validation Pipeline

Every commit to the `constitution/` repository runs the validation pipeline defined in `.github/workflows/validate-mrc.yml`. The pipeline executes four stages in sequence; any stage failure blocks the commit from being merged. The stages are:

1. **Stage 1 — CUE → JSON Schema.** CUE compilation: the `cue/constitution.cue` source is compiled to `schemas/*.schema.json`. If the CUE source does not compile, or if the generated JSON Schemas differ from the committed schemas/ (i.e., the committed schemas are stale), the stage fails. The check is a guarantee that the CUE source and the JSON Schemas are in sync.
2. **Stage 2 — Structural.** structural validation: every YAML file in the repository (except `meta.yaml`, the schemas themselves, and the CUE source) is validated against its class schema using `jsonschema 4.21`. Structural validation enforces required fields, types, enums, patterns, and `additionalProperties: false`. The validation map is documented in §6.1.
3. **Stage 3 — Semantic.** semantic validation: cross-field and cross-provision constraints are enforced by `validators/validate_mrc.py`. Semantic validation enforces rules like “if `severity = SEV-1`, `automation.policy_module` **MUST** be set” and “if `verifier = automated`, `automation` **MUST** be set”. The full semantic rule set is in §6.2.
4. **Stage 4 — Referential.** referential validation: every cross-reference (every ADR, test, evidence record, criterion, and code module cited in any provision) is resolved against the corresponding source of truth (the ADR repository, the test repository, the certification repository). Unresolved references fail the stage. The full referential rule set is in §6.3.
5. **Stage 5 — Version.** version compatibility validation: every provision's version field is checked for SemVer compliance, and the amended array is checked for monotonic version increase. The full version rule set is in §6.4.

```

# .github/workflows/validate-mrc.yml (excerpt)
name: validate-mrc
on: [pull_request, push]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:

```

```

- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with: { python-version: '3.12' }
- run: pip install jsonschema==4.21 pyyaml==6.0
- run: pip install cue==0.9 # or use the official cue binary
- name: Stage 1 — CUE → JSON Schema
  run: |
    cue export cue/constitution.cue \
      --out json \
      --force
  git diff --exit-code schemas/ # must be no-op
- name: Stage 2 — Structural validation
  run: python validators/validate_mrc.py --stage structural
- name: Stage 3 — Semantic validation
  run: python validators/validate_mrc.py --stage semantic
- name: Stage 4 — Referential validation
  run: python validators/validate_mrc.py --stage referential
- name: Stage 5 — Version validation
  run: python validators/validate_versions.py
- name: Generate derived artifacts
  run: |
    python generators/gen_constitution_vol1.py
    python generators/gen_constitution_vol2.py
    python generators/gen_playbook.py
    python generators/gen_traceability_matrix.py
    python generators/gen_certification_checklist.py
    python generators/gen_oscal.py
    python generators/gen_opa_bundle.py
python generators/gen_backstage_catalog.py

```

If Stage 5 passes, the generate-derived job runs every generator and commits the regenerated artifacts to constitution-built/. A pull request that changes any provision file therefore produces a diff of every affected derived artifact in the same PR, which is the primary review surface for constitutional amendments. Reviewers see both the YAML change and the resulting prose change in one place.

Chapter 3 — The Universal Provision Schema

3.1 The Eighteen-Field Schema

Every provision in the MRC — regardless of class — conforms to the universal provision schema. The universal schema has eighteen fields. Eight of these are required on every provision; the remaining ten are required only under conditions specified in the “Required” column. Class-specific schemas (Chapter 4) extend the universal schema with additional fields; no class-specific schema removes or renames a universal field.

Field	Type	Required	Description
id	string	REQUIRED	Stable, unique, immutable identifier for the provision. The id is the primary key by which the provision is referenced from ADRs, traceability matrices, tests, evidence records, certification criteria, and OPA policies.
class	enum	REQUIRED	Provision class. Determines which class-specific fields are required. The enum is closed: a provision MUST be exactly one of the ten classes enumerated in Chapter 4.
statement	string	REQUIRED	Single-sentence natural-language statement of the provision. Used by document generators as the lead line in every human-readable chapter. MUST be deterministic and self-contained; cross-references go in source_chapter.
source_chapter	string	REQUIRED	Citation of the originating constitutional chapter and section, in the form “Vol I/II Ch N §N.M”. Used by generators to emit the canonical provenance footer of every derived document.
severity	enum	REQUIRED	Severity class on violation. Drives alerting routing,

			paging policy, and release-gating behaviour. The enum is SEV-1, SEV-2, SEV-3.
owner	string	REQUIRED	Owning team, as listed in the Engineering Playbook team registry. The owner is the default assignee for incidents, amendments, and certification deficiencies; the owner MUST NOT be an individual.
predicate	string	REQUIRED for invariant, rule, transition	Formal predicate (for invariants) or condition (for rules) or guard (for transitions) in first-order logic notation. The predicate is the executable specification consumed by the OPA/Rego policy generator.
verification_method	string	REQUIRED	Procedural description of how the provision is verified. SHALL name the method (SQL query, monitor output, reconciliation report, manual inspection) and the success criterion. The field is consumed by the certification checklist generator and by the RCA.
required_evidence	object	REQUIRED	Evidence contract: the artifact type, format, maximum age, and storage location that demonstrate PASS. The full sub-schema is specified in §5.3.
verifier	enum	REQUIRED	Verifier role. Drives the separation-of-duties check: the verifier SHALL NOT be the implementer of the provision being verified. Enum: rca, qa, automated, auditor.
automation	object	REQUIRED when severity = SEV-1	Automation contract: the OPA/Rego policy module, the

			CI job that runs it, the runtime monitor, and the alert. The full sub-schema is specified in §5.2.
runtime_monitor	object	OPTIONAL	Runtime monitor contract: monitor id, cadence, implementation. Required for every invariant and every SEV-1 rule; optional otherwise.
slo	string	OPTIONAL	Reference to the SLO this provision contributes to, if any. Invariants are binary and SHALL NOT carry an SLO; rules and contracts often do.
certification_rule	string	REQUIRED	Reference to the certification criterion (CC-*) that gates this provision. Every provision SHALL have at least one certification criterion; the criterion is what makes the provision binary at release time.
traceability	object	REQUIRED	Traceability graph edges: architecture components, ADRs, database tables, API endpoints, code modules, storage artifacts, tests, evidence records. The full sub-schema is specified in §5.1.
version	string (SemVer)	REQUIRED	Semantic Versioning 2.0.0 of this provision. The version SHALL be incremented on every amendment: PATCH for editorial, MINOR for additive, MAJOR for breaking. The full SemVer rule is in §6.4.
amended	array	REQUIRED (may be empty on initial commit)	Amendment history: chronological array of {date, sponsor, rationale, version} entries. SHALL contain at least the initial-commit entry. The full sub-schema is

			specified in §5.4.
status	enum	REQUIRED	Lifecycle status. Enum: active, deprecated, retired. 'deprecated' MUST be paired with a successor reference; 'retired' MAY NOT be removed (see §8.4).

Schema Invariant S-1 (Closure)

The universal provision schema is closed under additionalProperties. A YAML file that contains a key not listed in the universal schema and not listed in the class-specific schema for its declared class SHALL fail structural validation. There are no ad-hoc fields.

3.2 Field-by-Field Reference

Each field of the universal schema is documented below in the order in which it appears in the schema. Every entry specifies type, required, description, valid values, an example, the validation rules that constrain the field, and the version history (the SemVer at which the field was introduced and at which it was last changed).

3.2.1 id

Type: string **Required:** REQUIRED

Stable, unique, immutable identifier for the provision. The id is the primary key by which the provision is referenced from ADRs, traceability matrices, tests, evidence records, certification criteria, and OPA policies.

Example: INV-I-02

Validation rules:

- MUST match the regex `^(INV-[A-Z]-\d{2}|(ARCH|DATA|PUB|EXT|OPS|PROC)-\d{3}|SUB-\d{2})|([a-z_]+|[A-Z]+\d{2,3})|EVT-\d{2}|SLO-\d{2}|LAT-[A-Z]+\d{3}|CC-.+)\$`.
- MUST be unique within the MRC repository (enforced by Stage 2 semantic validation).
- MUST equal the file name, in lowercase with conventional hyphenation.
- MUST NOT be changed after initial commit; rename requires a deprecation-and-successor workflow (§8.3).

Version history: Introduced 1.0.0. Pattern tightened in 1.0.0 to enforce class-prefixed format.

3.2.2 class

Type: enum **Required:** REQUIRED

Provision class. Determines which class-specific fields are required. The enum is closed: a provision **MUST** be exactly one of the ten classes enumerated in Chapter 4.

Example: *invariant*

Validation rules:

- **MUST** be exactly one of: invariant, rule, contract, field, event, slo, threshold, state, transition, criterion.
- **MUST** equal the directory the file resides in (invariants/, rules/, contracts/, fields/, events/, slo/, thresholds/, state-machine/, certification/criteria/).
- **MUST NOT** be changed after initial commit; class change requires retirement and re-creation.

Version history: *Introduced 1.0.0. Enum closed at 1.0.0 (ten classes).*

3.2.3 statement

Type: string **Required:** REQUIRED

Single-sentence natural-language statement of the provision. Used by document generators as the lead line in every human-readable chapter. **MUST** be deterministic and self-contained; cross-references go in `source_chapter`.

Example: *Exactly one DOI per published article version.*

Validation rules:

- **MUST** be a single sentence (one terminal period or equivalent).
- **MUST** be between 8 and 240 characters inclusive.
- **MUST NOT** contain RFC 2119 keywords (**MUST**, **SHALL**, etc.) — those belong in the predicate.
- **MUST** be self-contained; cross-references go in `source_chapter` and traceability.

Version history: *Introduced 1.0.0. Length constraint tightened in 1.0.0 (8–240).*

3.2.4 source_chapter

Type: string **Required:** REQUIRED

Citation of the originating constitutional chapter and section, in the form “Vol I/II Ch N §N.M”. Used by generators to emit the canonical provenance footer of every derived document.

Example: *Vol II Ch 26 §26.1*

Validation rules:

- **MUST** match the regex `^Vol (I|II) Ch \d+( §\d+\.\d+)?$`.
- **MUST** reference an existing chapter in the prose Constitution.
- **MUST NOT** be a free-form citation (no URLs, no prose); URLs go in traceability.

Version history: *Introduced 1.0.0. Pattern enforced since 1.0.0.*

3.2.5 severity

Type: enum **Required:** REQUIRED

Severity class on violation. Drives alerting routing, paging policy, and release-gating behaviour. The enum is SEV-1, SEV-2, SEV-3.

Example: SEV-1

Validation rules:

- MUST be one of: SEV-1, SEV-2, SEV-3.
- If SEV-1, the automation field MUST be present and the automation.policy_module MUST be set (enforced by Stage 3 semantic validation).
- MUST NOT be reduced (SEV-1 → SEV-2) without a recorded amendment with CSA sponsor.

Version history: Introduced 1.0.0. Enum {SEV-1, SEV-2, SEV-3} since 1.0.0.

3.2.6 owner

Type: string **Required:** REQUIRED

Owning team, as listed in the Engineering Playbook team registry. The owner is the default assignee for incidents, amendments, and certification deficiencies; the owner MUST NOT be an individual.

Example: Platform Engineering

Validation rules:

- MUST be a team name as listed in the Playbook team registry.
- MUST NOT be an individual's name or email address.
- SHOULD match the team listed in the source_chapter's Responsibilities subsection.

Version history: Introduced 1.0.0.

3.2.7 predicate

Type: string **Required:** REQUIRED for invariant, rule, transition

Formal predicate (for invariants) or condition (for rules) or guard (for transitions) in first-order logic notation. The predicate is the executable specification consumed by the OPA/Rego policy generator.

Example: $\forall v : \text{Published}(v) \rightarrow \exists! d : \text{HasDOI}(v, d) \wedge \text{Registered}(d)$

Validation rules:

- REQUIRED when class = invariant, rule, or transition.
- SHOULD use first-order logic notation ($\forall$, $\exists$, $\wedge$, $\vee$, $\rightarrow$, $\neg$) for invariants; modal/temporal operators ($\Box$, $\Diamond$) for rules.
- MUST be parseable by the OPA/Rego policy generator (generators/gen_opa_bundle.py) without manual rewriting.
- MUST be unique within the MRC repository (two provisions MUST NOT share an identical predicate).

Version history: *Introduced 1.0.0. Requiredness promoted to class-conditional in 1.0.0.*

3.2.8 verification_method

Type: string **Required:** REQUIRED

Procedural description of how the provision is verified. SHALL name the method (SQL query, monitor output, reconciliation report, manual inspection) and the success criterion. The field is consumed by the certification checklist generator and by the RCA.

Example: *SQL: COUNT(DISTINCT dois WHERE article_version_id = V) = 1 for every published V; reconciliation against Crossref REST API.*

Validation rules:

- MUST be at least 8 characters.
- MUST name the verification type (SQL query, monitor output, reconciliation report, manual inspection) and the success criterion.
- If verifier = automated, MUST be expressible as a Rego policy (the policy module is the executable form of this method).

Version history: *Introduced 1.0.0. Minimum length 8 since 1.0.0.*

3.2.9 required_evidence

Type: object **Required:** REQUIRED

Evidence contract: the artifact type, format, maximum age, and storage location that demonstrate PASS. The full sub-schema is specified in §5.3.

Example: *type: reconciliation_report; format: csv; max_age: 1h.*

Validation rules:

- MUST be an object conforming to the Evidence sub-schema (§5.3).
- MUST specify type, format, max_age, and storage.
- storage MUST begin with the prefix certification-repository/.
- max_age MUST match the regex `^\d+(s|m|h|d|w)$` and SHOULD match the runtime_monitor cadence.

Version history: *Introduced 1.0.0.*

3.2.10 verifier

Type: enum **Required:** REQUIRED

Verifier role. Drives the separation-of-duties check: the verifier SHALL NOT be the implementer of the provision being verified. Enum: rca, qa, automated, auditor.

Example: *automated*

Validation rules:

- MUST be one of: rca, qa, automated, auditor.
- If automated, the automation field MUST be present.
- MUST NOT be the same as the owner of any code module cited in traceability.code (separation of duties, enforced by Stage 3).

Version history: *Introduced 1.0.0.*

3.2.11 automation

Type: object **Required:** REQUIRED when severity = SEV-1

Automation contract: the OPA/Rego policy module, the CI job that runs it, the runtime monitor, and the alert. The full sub-schema is specified in §5.2.

Example: *policy_module: policies/INV-I-02.rego; ci_job: verify-invariants.yml.*

Validation rules:

- REQUIRED when severity = SEV-1 OR verifier = automated.
- policy_module MUST reference an existing file under policies/ (enforced by Stage 4 referential validation).
- ci_job MUST reference an existing workflow under .github/workflows/.
- monitor and alert MUST be unique within the MRC repository.

Version history: *Introduced 1.0.0. Requiredness promoted to SEV-1/automated-conditional in 1.0.0.*

3.2.12 runtime_monitor

Type: object **Required:** OPTIONAL

Runtime monitor contract: monitor id, cadence, implementation. Required for every invariant and every SEV-1 rule; optional otherwise.

Example: *id: INV-MON-I-02; cadence: hourly.*

Validation rules:

- REQUIRED when class = invariant.
- cadence MUST be one of: realtime, 1m, 5m, 15m, hourly, daily, weekly.
- cadence MUST be no slower than the max_age of required_evidence (a 1-hour monitor cannot back a 5-minute evidence).
- implementation MUST reference a deployable component (a Kubernetes Deployment, a CloudWatch metric, etc.).

Version history: *Introduced 1.0.0. Requiredness promoted to invariant-conditional in 1.0.0.*

3.2.13 slo

Type: string **Required:** OPTIONAL

Reference to the SLO this provision contributes to, if any. Invariants are binary and SHALL NOT carry an SLO; rules and contracts often do.

Example: SLO-01

Validation rules:

- OPTIONAL on rules, contracts, and thresholds.
- MUST NOT be set on invariants (invariants are binary, not SLO-tracked).
- MUST reference an existing SLO provision (slos/SLO-NN-*.yaml).

Version history: *Introduced 1.0.0.*

3.2.14 certification_rule

Type: string **Required:** REQUIRED

Reference to the certification criterion (CC-*) that gates this provision. Every provision SHALL have at least one certification criterion; the criterion is what makes the provision binary at release time.

Example: CC-INV-I-02

Validation rules:

- MUST match the regex ^CC-.
- MUST reference an existing criterion file (certification/criteria/CC-*.yaml).
- MAY be a single string or a list of strings (a provision can be gated by multiple criteria).

Version history: *Introduced 1.0.0.*

3.2.15 traceability

Type: object **Required:** REQUIRED

Traceability graph edges: architecture components, ADRs, database tables, API endpoints, code modules, storage artifacts, tests, evidence records. The full sub-schema is specified in §5.1.

Example: *adr: [ADR-007]; code: [services/doi-minter/].*

Validation rules:

- MUST be an object conforming to the Traceability sub-schema (§5.1).
- At least one of architecture, adr, code, or test MUST be non-empty.
- Every ADR cited MUST exist in the ADR repository.
- Every test cited MUST exist in the test repository and start with TEST-.
- Every evidence record cited MUST exist in the certification repository and start with EVID-.

Version history: *Introduced 1.0.0.*

3.2.16 version

Type: string (SemVer) **Required:** REQUIRED

Semantic Versioning 2.0.0 of this provision. The version SHALL be incremented on every amendment: PATCH for editorial, MINOR for additive, MAJOR for breaking. The full SemVer rule is in §6.4.

Example: 1.0.0

Validation rules:

- MUST match the regex `^\d+\.\d+\.\d+$` (SemVer 2.0.0).
- MUST be strictly greater than the version in the most recent entry of amended (enforced by Stage 5).
- MUST be incremented on every amendment: PATCH for editorial, MINOR for additive, MAJOR for breaking.

Version history: *Introduced 1.0.0. SemVer pattern enforced since 1.0.0.*

3.2.17 amended

Type: array **Required:** REQUIRED (may be empty on initial commit)

Amendment history: chronological array of {date, sponsor, rationale, version} entries. SHALL contain at least the initial-commit entry. The full sub-schema is specified in §5.4.

Example: `[[date: 2026-08-09, sponsor: CSA, version: 1.0.0]]`

Validation rules:

- MUST be an array with at least one entry.
- Each entry MUST conform to the Amendment sub-schema (§5.4).
- Entries MUST be in chronological order (date ascending).
- Versions across entries MUST be strictly monotonic increasing.
- The most recent entry's version MUST equal the top-level version field.

Version history: *Introduced 1.0.0. minItems = 1 enforced since 1.0.0.*

3.2.18 status

Type: enum **Required:** REQUIRED

Lifecycle status. Enum: active, deprecated, retired. 'deprecated' MUST be paired with a successor reference; 'retired' MAY NOT be removed (see §8.4).

Example: *active*

Validation rules:

- MUST be one of: active, deprecated, retired.
- If deprecated, the provision MUST carry successor and sunset_date fields (CUE enforces this).
- If retired, the provision MUST remain in the repository; removal requires a Constitution amendment per Vol II Ch 36.
- Retired provisions MUST NOT be referenced by any active provision's traceability (enforced by Stage 3).

Version history: *Introduced 1.0.0. Enum {active, deprecated, retired} since 1.0.0.*

3.3 The JSON Schema 2020-12 Definition

The universal provision schema is defined in JSON Schema 2020-12. The schema is generated from the CUE source (§3.4) by the cue export command, committed to schemas/provision.schema.json, and validated for freshness on every CI run. The committed form is:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://opuspublica.com/schemas/mrc/1.0.0/provision.schema.json",
  "title": "Universal Provision Schema",
  "description": "The closed schema every provision in the MRC conforms to.",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "id", "class", "statement", "source_chapter", "severity",
    "owner", "verification_method", "required_evidence",
    "verifier", "certification_rule", "traceability",
    "version", "amended", "status"
  ],
  "properties": {
    "id": {
      "type": "string",
      "pattern": "^(INV-[A-Z]-\\d{2})|(ARCH|DATA|PUB|EXT|OPS|PROC)-\\d{3}|SUB-\\d{2}|[a-z_]+|[A-Z]+-\\d{2,3}|EVT-\\d{2}|SLO-\\d{2}|LAT-[A-Z]+-\\d{3}|CC-\\.+)$",
      "description": "Stable, unique, immutable provision identifier."
    },
    "class": {
      "type": "string",
      "enum": ["invariant", "rule", "contract", "field", "event", "slo", "threshold", "state", "transition", "criterion"]
    },
    "statement": {
      "type": "string",
      "minLength": 8,
      "maxLength": 240,
      "source_chapter": {
        "type": "string",
        "pattern": "^Vol (I|II) Ch \\d+( \\u00a7\\d+\\.\\d+)?$",
        "severity": {
          "type": "string",
          "enum": ["SEV-1", "SEV-2", "SEV-3"]
        },
        "owner": {
          "type": "string",
          "minLength": 2,
          "maxLength": 80,
          "predicate": {
            "type": "string"
          },
          "verification_method": {
            "type": "string",
            "minLength": 8,
            "required_evidence": {
              "$ref": "evidence.schema.json",
              "verifier": {
                "type": "string",
                "enum": ["rca", "qa", "automated", "auditor"]
              },
              "automation": {
                "$ref": "automation.schema.json",
                "runtime_monitor": {
                  "type": "object",

```

```

        "additionalProperties": false,
        "required": ["id", "cadence", "implementation"],
        "properties": {
            "id": {"type": "string"},
            "cadence": {"type": "string",
                "enum": ["realtime", "1m", "5m", "15m",
                    "hourly", "daily", "weekly"]},
            "implementation": {"type": "string"}
        },
        "slo": {"type": ["string", "null"]},
        "certification_rule": {"type": "string",
            "pattern": "^CC-"},
        "traceability": {"$ref": "traceability.schema.json",
            "version": {"type": "string",
                "pattern": "^\\d+\\.\\d+\\.\\d+$"},
            "amended": {
                "type": "array",
                "minItems": 1,
                "items": {"$ref": "amendment.schema.json"}
            },
            "status": {"type": "string",
                "enum": ["active", "deprecated", "retired"]}
        },
        "allOf": [
            {
                "if": {"properties": {"class": {"const": "invariant"}}},
                "then": {"required": ["predicate", "runtime_monitor"]}
            },
            {
                "if": {"properties": {"severity": {"const": "SEV-1"}}},
                "then": {"required": ["automation"]}
            },
            {
                "if": {"properties": {"verifier": {"const": "automated"}}},
                "then": {"required": ["automation"]}
            }
        ]
    }

```

The schema declares three `allOf` branches that express cross-field constraints at the structural level: an invariant provision **MUST** carry a predicate and a `runtime_monitor`; a SEV-1 provision **MUST** carry an automation block; an automated-verifier provision **MUST** carry an automation block. These three constraints are necessary but not sufficient for full semantic validation — the deeper constraints (e.g., “the `automation.policy_module` file **MUST** exist in `policies/`”) are referential and enforced by `validators/validate_mrc.py` at Stage 4 (§6.3).

3.4 The CUE Source Definition

JSON Schema 2020-12 is sufficiently expressive for structural validation but cannot express several constraints that the MRC requires: closed unions with conditional fields, type-level references between fields, and conditional defaults. The MRC therefore authors the schema in CUE and compiles it to JSON Schema for runtime use. The CUE source is the authoring form; the JSON Schema is the compiled form. The pattern mirrors a compiler: authors write in the high-level language; the runtime executes the compiled form. The CUE source is:

```
//
package constitution/cue/constitution.cue
package constitution

import "list"

// Provision is the universal schema. Closed under additionalProperties
// by CUE's default closed-struct semantics.
#Provision: {
    // Required on every provision
    id: string & =~"^(INV-[A-Z]-\d{2}|(ARCH|DATA|PUB|EXT|OPS|PROC)-\d{3}|
SUB-\d{2})[a-z_]+|[A-Z]+\d{2,3}|EVT-\d{2}|SLO-\d{2}|LAT-[A-Z]+\d{3}|CC-.+)$"
    class: "invariant" | "rule" | "contract" | "field" | "event" | "slo" | "threshold" | "state"
    | "transition" | "criterion"
    statement: string & len >= 8 & len <= 240
    source_chapter: string & =~"^Vol (I|II) Ch \d+( \u00a7\d+\.\d+)?$"
    severity: "SEV-1" | "SEV-2" | "SEV-3"
    owner: string & len >= 2 & len <= 80
    verification_method: string & len >= 8
    required_evidence: #Evidence
    verifier: "rca" | "qa" | "automated" | "auditor"
    certification_rule: string & =~"^CC-"
    traceability: #Traceability
    version: string & =~"^\\d+\\.\\d+\\.\\d+$"
    amended: [...#Amendment] & len >= 1
    status: "active" | "deprecated" | "retired"

    // Conditional
    predicate?: string
    automation?: #Automation
    runtime_monitor?: #RuntimeMonitor
    slo?: string | null

    // Class-conditional enforcement (expressed natively in CUE)
    if class == "invariant" {
        predicate: string // promoted to required
        runtime_monitor: #RuntimeMonitor
    }
    if severity == "SEV-1" {
```

```

        automation: #Automation
      }
      if verifier == "automated" {
        automation: #Automation
      }
      if status == "deprecated" {
        successor: string & =~"^(INV|ARCH|DATA|PUB|EXT|OPS|PROC|SUB|EVT|SLO|LAT)-"
        sunset_date: string & =~"^\\d{4}-\\d{2}-\\d{2}$"
      }
    }

  #Evidence: {
    type: string // e.g., reconciliation_report
    format: "csv" | "json" | "xml" | "pdf" | "png" | "log"
    max_age: string & =~"^((\\d+(s|m|h|d|w)))$"
    storage: string & =~"^certification-repository/"
  }

  #Automation: {
    policy_module?: string & =~"^policies/.+\\.rego$"
    ci_job?: string
    monitor?: string
    alert?: string
  }

  #RuntimeMonitor: {
    id: string
    cadence: "realtime" | "1m" | "5m" | "15m" | "hourly" | "daily" | "weekly"
    implementation: string
  }

  #Traceability: {
    architecture?: [...string]
    adr?: [...string] & =~"^ADR-\\d{3}-"
    database?: [...string]
    api?: [...string]
    code?: [...string]
    storage?: [...string]
    test?: [...string] & =~"^TEST-"
    evidence?: [...string] & =~"^EVID-"
  }

  #Amendment: {
    date: string & =~"^\\d{4}-\\d{2}-\\d{2}$"
    sponsor: string
    rationale: string & len >= 8
    version: string & =~"^\\d+\\.\\d+\\.\\d+$"
  }

```

```
// SemVer monotonicity: every amendment's version MUST strictly increase.  
// Enforced by validate_versions.py (Stage 5) because CUE cannot express  
// order across list elements natively.
```

CUE's type system expresses the cross-field constraints (if class = invariant, predicate is required) directly, where JSON Schema requires the more verbose if/then/else form. CUE also provides compile-time evaluation (the cue export command produces the JSON Schema), open/closed union control via the # sigil, and default value propagation. The trade-off is that CUE is a younger language than JSON Schema with a smaller tooling ecosystem; the MRC uses CUE only for authoring and compiles to JSON Schema for runtime use by the wider Python and JavaScript ecosystems.

Chapter 4 — Provision Class Schemas

Each of the ten provision classes extends the universal schema with class-specific fields. This chapter documents, for each class, the class-specific fields, the JSON Schema 2020-12 definition, and two or three worked examples drawn from the actual MRC content. The class-specific fields are additive: a class schema is the universal schema plus the class-specific fields, with the class field constrained to the single enum value identifying the class.

4.1 Invariants (class: invariant)

Invariants are the constitutional guarantees that, if violated, constitute scholarly-record corruption. There are 38 invariants, organized into six categories (identity, immutability, resolvability, completeness, legality, auditability, conservation). Every invariant is SEV-1 by definition; an invariant that is not SEV-1 is, by definition, not an invariant.

Attribute	Value
Class enum value	invariant
Provision count	38
Source chapter	Vol II Ch 26
Class-specific fields	predicate (formal, REQUIRED), violation_response, monitor
Schema file	schemas/invariant.schema.json

4.1.1 Class-Specific Fields

Field	Type	Required	Description
predicate	string	REQUIRED (universal)	Formal first-order logic predicate. The executable specification consumed by the OPA/Rego policy generator.
violation_response	object	REQUIRED	Response contract on violation: the alert severity, the paging policy, the immediate remediation, and the post-incident review requirement.
monitor	object	REQUIRED (alias for runtime_monitor)	The runtime monitor that detects violations. Aliases the universal

			runtime_monitor field for backward readability with the Certification Checklist convention.
--	--	--	---

4.1.2 Example: INV-I-02 (Exactly one DOI per published article version)

```
# constitution/invariants/identity/INV-I-02.yaml
id: INV-I-02
class: invariant
statement: "Exactly one DOI per published article version."
source_chapter: "Vol II Ch 26 \u00a726.1"
severity: SEV-1
owner: Platform Engineering
predicate: "\u2200 v : Published(v) \u2192 \u2203! d : HasDOI(v, d) \u2227 Registered(d)"
verification_method: >-
  SQL: COUNT(DISTINCT dois WHERE article_version_id = V) = 1 for every
  published V; AND COUNT(article_versions WHERE doi = D) = 1 for every
  registered D. Crossref REST API reconciliation confirms DOI resolves
  and metadata matches.
required_evidence:
  type: reconciliation_report
  format: csv
  max_age: 1h
  storage: certification-repository/invariants/INV-I-02/
verifier: automated
automation:
  ci_job:
    policy_module: policies/INV-I-02.rego
    .github/workflows/verify-invariants.yml [INV-I-02]
    monitor: INV-MON-I-02
    alert: INV-I-02-violation
runtime_monitor:
  id: INV-MON-I-02
  cadence: hourly
  implementation: reconciliation-job-python
violation_response:
  alert_severity: SEV-1
  paging_policy: page-on-call-immediate
  immediate_remediation: "Quarantine affected article version; re-mint DOI if missing; de-
  duplicate if multiple."
  post_incident_review: REQUIRED
slo: null
certification_rule: CC-INV-I-02
traceability:
  architecture: [ARCH-publication-service, ARCH-doi-minter]
  adr: [ADR-007-doi-minting-strategy]
  database: [dois, article_versions]
```

```

      api: ["POST /api/v1/articles/{id}/mint-doi"]
      code: [services/doi-minter/, services/publication/guard.py]
      storage: []
      test: [TEST-INV-I-02-uniqueness, TEST-INV-I-02-crossref-reconciliation]
      evidence: [EVID-2026-INV-I-02-latest]
version: 1.0.0
amended:
  -
    date: 2026-08-09
    sponsor: CSA
    rationale: "Initial encoding of Vol II Ch 26 \u00a726.1 as MRC."
    version: 1.0.0
status: active

```

4.1.3 Example: INV-M-01 (Immutability of published article versions)

```

# constitution/invariants/immutability/INV-M-01.yaml
id: INV-M-01
class: invariant
statement: "A published article version is immutable."
source_chapter: "Vol II Ch 26 \u00a726.2"
severity: SEV-1
owner: Platform Engineering
predicate: "\u2200 v : Published(v) \u2192 \u00ac\u2203 t : Mutates(v, t)"
verification_method: >-
  Hash reconciliation: SHA-256 of every published article version is
  recomputed hourly and compared against the hash recorded at
  publication time. Any mismatch is a violation.
required_evidence:
  type: hash_reconciliation_report
  format: json
  max_age: 1h
  storage: certification-repository/invariants/INV-M-01/
verifier: automated
automation:
  policy_module: policies/INV-M-01.rego
  ci_job: .github/workflows/verify-invariants.yml [INV-M-01]
  monitor: INV-MON-M-01
  alert: INV-M-01-violation
runtime_monitor:
  id: INV-MON-M-01
  cadence: hourly
  implementation: hash-reconciliation-job
violation_response:
  alert_severity: SEV-1
  paging_policy: page-on-call-immediate
  immediate_remediation: "Revert unauthorized mutation; file SEV-1 incident; notify editor."
  post_incident_review: REQUIRED
slo: null

```



```

certification_rule: CC-INV-M-01
traceability:
  architecture: [ARCH-storage-service, ARCH-publication-service]
  adr: [ADR-014-content-addressed-storage]
  database: [article_versions, content_hashes]
  api: []
  code: [services/storage/hash_guard.py]
  storage: [s3://opuspublica-content/]
  test: [TEST-INV-M-01-hash-immutability]
  evidence: [EVID-2026-INV-M-01-latest]
version: 1.0.0
amended:
  -
  date: 2026-08-09
  sponsor: CSA
  rationale: "Initial encoding of Vol II Ch 26 \u00a726.2 as MRC."
  version: 1.0.0
status: active

```

4.1.4 Example: INV-A-03 (Audit chain integrity)

```

# constitution/invariants/auditability/INV-A-03.yaml
id: INV-A-03
class: invariant
statement: "The audit chain is append-only and hash-linked."
source_chapter: "Vol II Ch 26 \u00a726.6"
severity: SEV-1
owner: Security Engineering
predicate: "\u2200 n \u2208 AuditChain : PrevHash(n) = Hash(n-1) \u2227 \u00ac Mutates(n, m)"
verification_method: >-
  Walk the audit chain from genesis to latest; recompute SHA-256 of
  every node and verify it equals the next node's prev_hash. Any break
  is a violation.
required_evidence:
  type: chain_walk_report
  format: json
  max_age: 1h
  storage: certification-repository/invariants/INV-A-03/
verifier: automated
automation:
  policy_module: policies/INV-A-03.rego
  ci_job: .github/workflows/verify-invariants.yml [INV-A-03]
  monitor: INV-MON-A-03
  alert: INV-A-03-violation
runtime_monitor:
  id: INV-MON-A-03
  cadence: hourly
  implementation: audit-chain-walker

```

```

violation_response:
    alert_severity: SEV-1
    paging_policy: page-ciso-immediate
    immediate_remediation: "Freeze audit subsystem; preserve forensic snapshot; engage CISO."
    post_incident_review: REQUIRED
slo: null
certification_rule: CC-INV-A-03
traceability:
    architecture: [ARCH-audit-service]
    adr: [ADR-021-audit-chain-design]
    database: [audit_log]
    api: []
    code: [services/audit/chain.py]
    storage: []
    test: [TEST-INV-A-03-chain-integrity]
    evidence: [EVID-2026-INV-A-03-latest]
version: 1.0.0
amended:
    -
    date: 2026-08-09
    sponsor: CISO
    rationale: "Initial encoding of Vol II Ch 26 \u00a726.6 as MRC."
    version: 1.0.0
status: active

```

4.2 Rules (class: rule)

Rules are the non-negotiable engineering standards — architecture, data, publication, external, operations, process — that bind every implementation. There are 47 rules. Rules differ from invariants in that a rule violation is a defect (the implementation is wrong) rather than corruption (the record is wrong); the consequence is therefore release-blocking rather than incident-triggering, but the rule does not retroactively corrupt already-published material.

Attribute	Value
Class enum value	rule
Provision count	47
Source chapter	Vol I Ch 3
Class-specific fields	enforcement_layer, enforcement_mechanism, detection
Schema file	schemas/rule.schema.json

4.2.1 Class-Specific Fields

Field	Type	Required	Description
enforcement_layer	enum	REQUIRED	Where the rule is enforced: ci (build time), deploy (deployment gate), runtime (request path), audit (post-hoc).
enforcement_mechanism	string	REQUIRED	The mechanism: the OPA policy name, the CI check name, the runtime guard module, or the audit script.
detection	object	REQUIRED	How a violation is detected: the detection point (ci, runtime, audit), the detection latency, and the alert routing.
predicate	string	REQUIRED (universal)	Formal condition. For rules, the predicate is a modal or temporal logic statement (e.g., $\Box$ every request).

4.2.2 Example: ARCH-001 (Single publication service)

```
# constitution/rules/architecture/ARCH-001.yaml
id: ARCH-001
class: rule
statement: "Exactly one service is responsible for publication state transitions."
source_chapter: "Vol I Ch 3 \u00a73.1"
severity: SEV-1
owner: Platform Engineering
predicate: "\u25c1 req : MutatesPublicationState(req) \u2192 Handler(req) = PublicationService"
verification_method: >-
  Static analysis of the service mesh: every route that calls a
  publication-state mutation endpoint MUST resolve to the
  publication service. Verified by service-mesh graph query.
required_evidence:
  type: service_mesh_graph_query
  format: json
  max_age: 24h
  storage: certification-repository/rules/ARCH-001/
verifier: automated
automation:
```

```

      policy_module: policies/ARCH-001.rego
ci_job: .github/workflows/verify-rules.yml [ARCH-001]
      monitor: RULE-MON-ARCH-001
      alert: ARCH-001-violation
enforcement_layer: ci
enforcement_mechanism: "opa-eval" service-mesh-graph.rego --input mesh.json"
detection:
      detection_point: ci
      detection_latency: 0s
      alert_routing: PR-block
slo: null
certification_rule: CC-ARCH-001
traceability:
  architecture: [ARCH-publication-service, ARCH-service-mesh]
  adr: [ADR-002-service-decomposition]
  database: []
  api: ["POST /api/v1/articles/{id}/publish"]
  code: [mesh/routes.yaml, services/publication/]
  storage: []
  test: [TEST-ARCH-001-mesh-isolation]
  evidence: [EVID-2026-ARCH-001-latest]
version: 1.0.0
amended:
  -
    date: 2026-08-09
    sponsor: CSA
  rationale: "Initial encoding of Vol I Ch 3 \u00a73.1 as MRC."
  version: 1.0.0
status: active

```

4.2.3 Example: PUB-001 (Every published article has exactly one DOI)

```

# constitution/rules/publication/PUB-001.yaml
id: PUB-001
class: rule
statement: "Every published article SHALL have exactly one DOI registered with Crossref."
source_chapter: "Vol I Ch 3 \u00a73.3"
severity: SEV-1
owner: Platform Engineering
predicate: "\u25c1 a : Published(a) \u2192 \u2203! d : HasDOI(a, d) \u2227 CrossrefRegistered(d)"
verification_method: >-
  Reconcile published articles against the Crossref REST API
  (api.crossref.org/works). Every published article MUST have
  exactly one DOI; every registered DOI MUST resolve and MUST
  return metadata matching the article record.
required_evidence:
  type: crossref_reconciliation_report
  format: csv

```

```

max_age: 24h
storage: certification-repository/rules/PUB-001/
verifier: automated
automation:
  policy_module: policies/PUB-001.rego
  ci_job: .github/workflows/verify-rules.yml [PUB-001]
  monitor: RULE-MON-PUB-001
  alert: PUB-001-violation
enforcement_layer: runtime
enforcement_mechanism: "publication-guard.py: assert has_single_doi(article) before state
transition"
detection:
  detection_point: runtime
  detection_latency: 0s
  alert_routing: SEV-1-page
slo: SLO-01
certification_rule: CC-PUB-001
traceability:
  architecture: [ARCH-publication-service, ARCH-doi-minter, ARCH-crossref-client]
  adr: [ADR-007-doi-minting-strategy, ADR-022-crossref-rest-migration]
  database: [articles, dois]
  api: ["POST /api/v1/articles/{id}/publish", "GET /api/v1/articles/{id}/doi"]
  code: [services/publication/guard.py, services/doi-minter/]
  storage: []
  test: [TEST-PUB-001-doi-uniqueness, TEST-PUB-001-crossref-reconciliation]
  evidence: [EVID-2026-PUB-001-latest]
version: 1.0.1
amended:
  -
    date: 2026-08-09
    sponsor: CSA
    rationale: "Initial encoding of Vol I Ch 3 \u00a73.3 as MRC."
    version: 1.0.0
  -
    date: 2026-08-30
    sponsor: Platform Engineering
    rationale: "Migrated Crossref verification from Event Data (sunset 2026-04-23) to REST
API per ADR-022."
    version: 1.0.1
status: active

```

4.2.4 Example: PROC-006 (Every PR SHALL be reviewed by an engineer other than the author)

```

# constitution/rules/process/PROC-006.yaml
id: PROC-006
class: rule
statement: "Every merged PR SHALL have at least one approval from an engineer other
than the author."
source Chapter: "Vol I Ch 3 \u00a73.6"
severity: SEV-2

```

```

owner: Engineering Operations
predicate: "\u25c1 pr : Merged(pr) \u2192 \u2203 r : Approved(r, pr) \u2227 Author(r) \u2260 Author(pr)"
verification_method: >-
  Query the GitHub API for every merged PR in the reporting period;
  verify that each has at least one approval from a non-author.
required_evidence:
  type: github_pr_audit_report
  format: csv
  max_age: 24h
  storage: certification-repository/rules/PROC-006/
verifier: automated
automation:
  policy_module: policies/PROC-006.rego
  ci_job: .github/workflows/verify-rules.yml [PROC-006]
  monitor: RULE-MON-PROC-006
  alert: PROC-006-violation
enforcement_layer: deploy
enforcement_mechanism: "GitHub branch protection: require pull request reviews before merge"
detection:
  detection_point: audit
  detection_latency: 24h
  alert_routing: engineering-ops-report
slo: null
certification_rule: CC-PROC-006
traceability:
  architecture: []
  adr: [ADR-031-branch-protection-policy]
  database: []
  api: ["GET /repos/{owner}/{repo}/pulls/{pull_number}/reviews"]
  code: [.github/branch-protection.yml]
  storage: []
  test: [TEST-PROC-006-approval-audit]
  evidence: [EVID-2026-PROC-006-latest]
version: 1.0.0
amended:
  -
  date: 2026-08-09
  sponsor: Engineering Operations
  rationale: "Initial encoding of Vol I Ch 3 \u00a73.6 as MRC."
  version: 1.0.0
status: active

```

4.3 System Contracts (class: contract)

System contracts are the per-subsystem interface specifications: the inputs a subsystem accepts, the outputs it produces, the authority it holds, the failure conditions it must handle, the dependencies it

consumes, the performance budget it must meet, the certification it must pass, the acceptance tests it must satisfy, and the runtime evidence it must emit. There are 20 system contracts, one per subsystem in the Opus Publica architecture.

Attribute	Value
Class enum value	contract
Provision count	20
Source chapter	Vol II Ch 27
Class-specific fields	inputs, outputs, authority, failure_conditions, dependencies, performance_budget, certification, acceptance_tests, runtime_evidence
Schema file	schemas/contract.schema.json

4.3.1 Class-Specific Fields

Field	Type	Required	Description
inputs	array	REQUIRED	List of input contracts: name, type, source, validation, idempotency.
outputs	array	REQUIRED	List of output contracts: name, type, destination, guarantee (at-least-once, exactly-once).
authority	object	REQUIRED	The authority held by the subsystem: databases it may write, external services it may call, secrets it may access.
failure_conditions	array	REQUIRED	Enumeration of failure conditions and the subsystem's response to each.
dependencies	array	REQUIRED	List of upstream subsystems and external services the subsystem depends on, with criticality (critical, degradable, optional).
performance_budget	object	REQUIRED	The latency, throughput, and error-rate budget the

			subsystem must meet, cross-referenced to the relevant threshold provisions.
certification	object	REQUIRED	The certification criteria that gate this contract, with the gating severity.
acceptance_tests	array	REQUIRED	List of acceptance test identifiers (TEST-*) that must pass before the contract is considered satisfied.
runtime_evidence	array	REQUIRED	List of runtime evidence artifacts the subsystem must emit (logs, metrics, traces) and their storage location.

4.3.2 Example: SUB-10 (DOI Minter)

```
# constitution/contracts/SUB-10-doi-minter.yaml
id: SUB-10
class: contract
statement: "The DOI Minter registers DOIs with Crossref for published article versions."
source_chapter: "Vol II Ch 27 \u00a727.10"
severity: SEV-1
owner: Platform Engineering
predicate: "\u25c1 v : Published(v) \u2227 \u00acHasDOI(v) \u2192 MintDOI(v) \u229d Registered(v, d)"
verification_method: >-
  Integration test: submit a published article version without a DOI;
  verify that the DOI Minter mints a DOI, registers it with Crossref,
  and writes the DOI back to the article version record within 30s.
required_evidence:
  type: integration_test_report
  format: json
  max_age: 24h
  storage: certification-repository/contracts/SUB-10/
verifier: qa
automation:
  ci_job:
    policy_module: policies/SUB-10.rego
    .github/workflows/verify-contracts.yml [SUB-10]
    monitor: CONTRACT-MON-SUB-10
    alert: SUB-10-violation
runtime_monitor:
```


id:	CONTRACT-MON-SUB-10
cadence:	5m
implementation:	doi-minter-health-check
slo:	SLO-10
certification_rule:	CC-SUB-10
traceability:	
architecture:	[ARCH-doi-minter]
adr:	[ADR-007-doi-minting-strategy, ADR-022-crossref-rest-migration]
database:	[dois, article_versions]
api:	["POST /api/v1/articles/{id}/mint-doi", "GET /api/v1/dois/{doi}"]
code:	[services/doi-minter/]
storage:	[]
test:	[TEST-SUB-10-mint-and-register, TEST-SUB-10-idempotency, TEST-SUB-10-retry-on-crossref-5xx]
evidence:	[EVID-2026-SUB-10-latest]
inputs:	
-	name: article_version_id
	type: UUID
	source: PublicationService
validation:	"exists in article_versions WHERE status = 'published'"
idempotency:	"if article_version_id already has a DOI, return existing DOI"
outputs:	
-	name: doi
type:	string (Crossref DOI)
	destination: PublicationService
	guarantee: exactly-once
authority:	
	databases_write: [dois]
external_services_call:	[Crossref REST API]
secrets_access:	[crossref-api-credentials]
failure_conditions:	
-	condition: "Crossref API returns 5xx"
response:	"retry with exponential backoff (max 5 attempts); if exhausted, enqueue for retry; emit alert"
-	condition: "Crossref API returns 4xx"
response:	"no retry; emit alert; do not write DOI back to article_versions"
-	condition: "Database write fails after Crossref registration"
response:	"compensating transaction: de-register DOI with Crossref; emit SEV-1 alert"
dependencies:	
-	subsystem: PublicationService
	criticality: critical
-	service: Crossref REST API
	criticality: critical
performance_budget:	
	p50_latency: 2s
	p95_latency: 5s
	p99_latency: 10s
throughput:	10 minting ops/s

	error_rate:	<	0.1%
threshold_refs:	[LAT-002-doi-mint,	THP-005-doi-mint]	
certification:	criteria:	[CC-SUB-10]	
	gating_severity:	SEV-1	
acceptance_tests:	-	TEST-SUB-10-mint-and-register	
	-	TEST-SUB-10-idempotency	
	-	TEST-SUB-10-retry-on-crossref-5xx	
	-	TEST-SUB-10-compensating-transaction	
runtime_evidence:	-	type:	structured_log
		format:	json
		storage:	logs/doi-minter/
	-	type:	prometheus_metric
		format:	prometheus
		storage:	prometheus/
	-	type:	opentelemetry_trace
		format:	otlp
		storage:	jaeger/
version:			1.0.0
amended:	-	date:	2026-08-09
		sponsor:	CSA
rationale:	"Initial encoding of Vol II Ch 27 \u00a727.10 as MRC."	version:	1.0.0
status:	active		

4.4 Canonical Fields (class: field)

Canonical fields are the per-entity field specifications: for each field of each entity (article, author, journal, etc.), the canonical field provision specifies the database table and column, the storage path (for content-addressed fields), the producing subsystem, the consuming subsystems, the validation rules, the mutability class, and the migration policy. There are 276 canonical fields.

Attribute	Value
Class enum value	field
Provision count	276
Source chapter	Vol II Ch 28
Class-specific fields	db_table, db_column, storage_path, producer, consumer, validation, mutability, migration
Schema file	schemas/field.schema.json

4.4.1 Class-Specific Fields

Field	Type	Required	Description
db_table	string	REQUIRED	Database table containing the field.
db_column	string	REQUIRED	Database column. Composite fields use dot notation (e.g., metadata.jats.title).
storage_path	string	OPTIONAL	For content-addressed fields, the storage URI template (e.g., s3://opuspublica-content/{sha256}.pdf).
producer	string	REQUIRED	The subsystem that writes the field.
consumer	array	REQUIRED	The subsystems that read the field.
validation	object	REQUIRED	Validation contract: type, pattern, enum, range, foreign key.
mutability	enum	REQUIRED	Mutability class: immutable-on-publish, mutable-pre-publish, append-only, mutable.
migration	object	REQUIRED	Migration contract: backward-compatibility, forward-compatibility, default value, migration script reference.

4.4.2 Example: article_doi

```
# constitution/fields/article/article_doi.yaml
id: article_doi
class: field
statement: "The DOI assigned to an article version, registered with Crossref."
source_chapter: "Vol II Ch 28 \u00a728.1"
severity: SEV-1
owner: Platform Engineering
predicate: "\u25c1 v : Published(v) \u2192 \u2203! d : article_doi(v) = d \u2227 CrossrefRegistered(d)"
```

```

verification_method: >-
  SQL: article_doi IS NOT NULL for every published article version;
  Crossref REST API reconciliation confirms each article_doi resolves
        and metadata matches.
required_evidence:
  type: reconciliation_report
  format: csv
  max_age: 1h
  storage: certification-repository/fields/article_doi/
verifier: automated
automation:
  policy_module: policies/article_doi.rego
  ci_job: .github/workflows/verify-fields.yml [article_doi]
  monitor: FIELD-MON-article_doi
  alert: article_doi-violation
slo: null
certification_rule: CC-FIELD-article_doi
traceability:
  architecture: [ARCH-doi-minter, ARCH-publication-service]
  adr: [ADR-007-doi-minting-strategy]
  database: [article_versions.doi]
  api: ["GET /api/v1/articles/{id}/doi"]
  code: [services/doi-minter/, services/publication/]
  storage: []
  test: [TEST-FIELD-article_doi-not-null, TEST-FIELD-article_doi-crossref-resolves]
  evidence: [EVID-2026-FIELD-article_doi-latest]
db_table: article_versions
db_column: doi
storage_path: null
producer: SUB-10
consumer: [SUB-01-submission, SUB-04-search, SUB-12-preservation, SUB-15-export]
validation:
  type: string
  pattern: "^10\\.\\d{4,9}/.+ $" # Crossref DOI format
  foreign_key: "dois.doi"
mutability: immutable-on-publish
migration:
  backward_compatibility: FULL # existing DOIs never change
  forward_compatibility: FULL
  default_value: null
  migration_script: null # no migration ever required
version: 1.0.0
amended:
  date: 2026-08-09
  sponsor: CSA
  rationale: "Initial encoding of Vol II Ch 28 \u00a728.1 as MRC."
  version: 1.0.0
status: active

```

4.5 Events (class: event)

Events are the asynchronous notifications that flow through the platform's event bus. There are 34 events, each emitted by exactly one producer and consumed by one or more consumers. Every event specifies its payload schema, its ordering guarantee, its idempotency key, and the outbox table that ensures reliable delivery.

Attribute	Value
Class enum value	event
Provision count	34
Source chapter	Vol II Ch 29
Class-specific fields	producer, consumer, payload_schema, ordering, idempotency_key, outbox_table
Schema file	schemas/event.schema.json

4.5.1 Class-Specific Fields

Field	Type	Required	Description
producer	string	REQUIRED	The subsystem that emits the event (SUB-*).
consumer	array	REQUIRED	The subsystems that consume the event (SUB-*).
payload_schema	object	REQUIRED	JSON Schema of the event payload.
ordering	enum	REQUIRED	Ordering guarantee: per-key-ordering, global-ordering, unordered.
idempotency_key	string	REQUIRED	The payload field used as the idempotency key.
outbox_table	string	REQUIRED	The database table used as the transactional outbox.

4.5.2 Example: EVT-22 PublicationCompleted

```
# constitution/events/EVT-22-PublicationCompleted.yaml
id: EVT-22
class: event
statement: "Emitted when an article version completes publication (DOI minted, preserved,
```

```

indexed)."
source_chapter:      "Vol      II      Ch      29      \u00a729.22"
severity:            SEV-1
owner:              Platform      Engineering
predicate: "\u25c1 v : Published(v) \u2227 DOIRegistered(v) \u2227 Preserved(v) \u2192
Emit(EVT-22,
verification_method: >-
    Replay test: submit an article through the full publication
    pipeline; verify EVT-22 is emitted exactly once with the correct
    payload; verify every consumer received it within 60s.
required_evidence:
    type:            event_replay_report
    format:          json
    max_age:         24h
    storage:         certification-repository/events/EVT-22/
verifier:            automated
automation:
    policy_module:   policies/EVT-22.rego
    ci_job:          .github/workflows/verify-events.yml [EVT-22]
    monitor:         EVENT-MON-EVT-22
    alert:           EVT-22-violation
slo:                 SLO-01
certification_rule:  CC-EVT-22
traceability:
    architecture:    [ARCH-event-bus, ARCH-publication-service]
    adr:             [ADR-018-transactional-outbox]
    database:        [event_outbox]
    api:             []
    code:            [services/publication/event_emitter.py, services/event-bus/]
    storage:         []
    test: [TEST-EVT-22-replay, TEST-EVT-22-idempotency, TEST-EVT-22-consumer-lag]
    evidence:        [EVID-2026-EVT-22-latest]
producer:            SUB-01-submission
consumer:
    - SUB-04-search      # index the published article
    - SUB-12-preservation # trigger CLOCKSS/LOCKSS deposit
    - SUB-15-export      # push to ORCID, DOAJ
    - SUB-17-analytics   # update publication counters
payload_schema:
    type: object
    required: [event_id, article_version_id, doi, published_at]
    properties:
        event_id: { type: string, format: uuid }
        article_version_id: { type: string, format: uuid }
        doi: { type: string, pattern: "^10\\.\\d{4,9}\\.+ $" }
        published_at: { type: string, format: date-time }
        journal_id: { type: string, format: uuid }
        issue_id: { type: [string, null], format: uuid }

```

```

ordering:      per-key-ordering      #      ordered      by      article_version_id
idempotency_key:                                     event_id
outbox_table:                                     event_outbox
version:                                             1.0.0
amended:
-
date: 2026-08-09
sponsor: CSA
rationale: "Initial encoding of Vol II Ch 29 \u00a729.22 as MRC."
version: 1.0.0
status: active

```

4.6 SLOs (class: slo)

Service Level Objectives specify the reliability targets the platform commits to. There are 10 SLOs, each defined by an SLI formula, a target, a measurement window, an error budget, and the burn-rate alerts that fire when the budget is consumed too quickly.

Attribute	Value
Class enum value	slo
Provision count	10
Source chapter	Vol II Ch 32
Class-specific fields	sli_formula, target, window, error_budget, burn_rate_alerts
Schema file	schemas/slo.schema.json

4.6.1 Class-Specific Fields

Field	Type	Required	Description
sli_formula	string	REQUIRED	Service Level Indicator formula (PromQL or equivalent).
target	number	REQUIRED	Target reliability as a fraction in [0, 1] (e.g., 0.999 for 99.9%).
window	enum	REQUIRED	Measurement window: 5m, 1h, 6h, 24h, 7d, 28d, 90d.
error_budget	number	REQUIRED	Error budget as a fraction in [0, 1] (= 1 - target).
burn_rate_alerts	array	REQUIRED	List of burn-rate alert thresholds: window, burn

			rate, severity, action (page, ticket, freeze).
--	--	--	--

4.6.2 Example: SLO-01 Availability

```
# constitution/slos/SLO-01-availability.yaml
id: SLO-01
class: slo
statement: "The platform SHALL be available with 99.9% success rate over a 28-day window."
source_chapter: "Vol II Ch 32 \u00a732.1"
severity: SEV-2
owner: Platform Engineering
predicate: "\u25c1 t : SuccessRatio(window=28d, t) \u2265 0.999"
verification_method: >-
  Prometheus query: sum(rate(http_requests_total{status=~"2.."}[28d]))
  / sum(rate(http_requests_total[28d])) \u2265 0.999.
required_evidence:
  type: slo_evaluation_report
  format: json
  max_age: 1h
  storage: certification-repository/slos/SLO-01/
verifier: automated
automation:
  policy_module: policies/SLO-01.rego
  ci_job: .github/workflows/verify-slos.yml [SLO-01]
  monitor: SLO-MON-01
  alert: SLO-01-burn-rate
slo: null # this IS the SLO
certification_rule: CC-SLO-01
traceability:
  architecture: [ARCH-load-balancer, ARCH-api-gateway]
  adr: [ADR-025-slo-framework]
  database: []
  api: []
  code: [services/load-balancer/, services/api-gateway/]
  storage: []
  test: [TEST-SLO-01-burn-rate-simulation]
  evidence: [EVID-2026-SLO-01-latest]
sli_formula: "sum(rate(http_requests_total{status=~\"2..\"}[28d])) /
sum(rate(http_requests_total[28d]))"
target: 0.999
window: 28d
error_budget: 0.001
burn_rate_alerts:
  - window: 1h
    burn_rate: 14.4 # 1h burn consuming 30d budget
```


									severity:	SEV-1
			-						action:	page
			#	6h	burn	consuming	25%	of	28d	budget 6h
	burn_rate:	6.0							window:	
									severity:	SEV-2
									action:	page
			-						window:	24h
	burn_rate:	3.0	#	24h	burn	consuming	30%	of	28d	budget
									severity:	SEV-2
									action:	ticket
			-						window:	24h
	burn_rate:	1.0	#	24h	burn	consuming	10%	of	28d	budget
									severity:	SEV-3
									action:	freeze-release
version:										1.0.0
amended:										
			-						date:	2026-08-09
									sponsor:	CSA
	rationale:	"Initial encoding of Vol II Ch 32 \u00a7732.1 as MRC."								
									version:	1.0.0
status:	active									

4.7 Thresholds (class: threshold)

Thresholds are the per-operation performance budgets: for each operation class (latency, throughput, batch, queue, storage), the threshold specifies the p50, p95, p99, the hard limit (gating), and the soft warning (informational). There are 103 thresholds.

Attribute	Value
Class enum value	threshold
Provision count	103
Source chapter	Vol II Ch 31
Class-specific fields	budget_class, operation, p50, p95, p99, hard_limit, soft_warning
Schema file	schemas/threshold.schema.json

4.7.1 Class-Specific Fields

Field	Type	Required	Description
budget_class	enum	REQUIRED	Threshold class: latency, throughput, batch, queue, storage.

operation	string	REQUIRED	Operation identifier (e.g., api-read, doi-mint, batch-deposit).
p50	string	REQUIRED	50th-percentile budget (e.g., 100ms, 10/s, 5MB).
p95	string	REQUIRED	95th-percentile budget.
p99	string	REQUIRED	99th-percentile budget.
hard_limit	string	REQUIRED	Hard limit; crossing it gates the release.
soft_warning	string	REQUIRED	Soft warning; crossing it triggers an informational alert.

4.7.2 Example: LAT-001 API read latency

```
# constitution/thresholds/latency/LAT-001-api-read.yaml
id: LAT-001
class: threshold
statement: "API read latency SHALL be under 200ms p95."
source_chapter: "Vol II Ch 31 \u00a731.1"
severity: SEV-2
owner: Platform Engineering
predicate: "\u25c1 r : ApiRead(r) \u2192 Latency(r).p95 \u2264 200ms"
verification_method: >-
    Prometheus histogram: histogram_quantile(0.95,
    sum(rate(http_request_duration_seconds_bucket{verb="GET"}[5m]))
    by (le)) \u2264 0.200.
required_evidence:
    type: latency_percentile_report
    format: json
    max_age: 1h
    storage: certification-repository/thresholds/LAT-001/
verifier: automated
automation:
    policy_module: policies/LAT-001.rego
    ci_job: .github/workflows/verify-thresholds.yml [LAT-001]
    monitor: THRESHOLD-MON-LAT-001
    alert: LAT-001-violation
slo: SLO-01
certification_rule: CC-LAT-001
traceability:
    architecture: [ARCH-api-gateway, ARCH-load-balancer]
    adr: [ADR-026-latency-budgets]
    database: []
```

```

api: ["GET /api/v1/articles/{id}", "GET /api/v1/journals/{id}", "GET /api/v1/issues/{id}"]
code: [services/api-gateway/]
storage: []
test: [TEST-LAT-001-load-test, TEST-LAT-001-tail-latency]
evidence: [EVID-2026-LAT-001-latest]
budget_class: latency
operation: api-read
p50: 50ms
p95: 200ms
p99: 500ms
hard_limit: 500ms # gate release if exceeded
soft_warning: 250ms # informational alert
version: 1.0.0
amended:
- date: 2026-08-09
  sponsor: CSA
  rationale: "Initial encoding of Vol II Ch 31 \u00a731.1 as MRC."
  version: 1.0.0
status: active

```

4.8 State Machine States (class: state)

The publication state machine has 13 states. Each state provision specifies the invariants that hold while in the state, the preconditions that must be true to enter the state, and the postconditions that must be true to leave the state.

Attribute	Value
Class enum value	state
Provision count	13
Source chapter	Vol I Ch 11
Class-specific fields	invariants, preconditions, postconditions
Schema file	schemas/state.schema.json

4.8.1 Class-Specific Fields

Field	Type	Required	Description
invariants	array	REQUIRED	List of INV-* identifiers whose predicates MUST hold while in this state.
preconditions	array	REQUIRED	List of conditions that MUST be true to enter this state (formal predicates).

postconditions	array	REQUIRED	List of conditions that MUST be true to leave this state (formal predicates).
----------------	-------	----------	---

4.8.2 Example: Published state

```
# constitution/state-machine/states.yaml (excerpt: Published)
-
  id: STATE-Published
  class: state
  statement: "The article version is published and immutable."
  source_chapter: "Vol I Ch 11" \u00a711.5"
  severity: SEV-1
  owner: Platform Engineering
  predicate: "State(article_version) = Published"
  verification_method: >-
    Query article_versions WHERE state = 'Published'; verify each
    has a DOI, a preservation deposit, and an immutable hash.
  required_evidence:
    type: state_inventory_report
    format: csv
    max_age: 1h
  storage: certification-repository/states/STATE-Published/
  verifier: automated
  automation:
    policy_module: policies/STATE-Published.rego
  ci_job: .github/workflows/verify-state-machine.yml [STATE-Published]
  monitor: STATE-MON-Published
  alert: STATE-Published-violation
  slo: null
  certification_rule: CC-STATE-Published
  traceability:
    architecture: [ARCH-publication-service]
    adr: [ADR-002-service-decomposition]
    database: [article_versions.state]
  api: ["GET /api/v1/articles/{id}/state"]
  code: [services/publication/state_machine.py]
  storage: []
  test: [TEST-STATE-Published-invariants-hold]
  evidence: [EVID-2026-STATE-Published-latest]
  invariants: [INV-I-02, INV-M-01, INV-R-01, INV-C-01] # DOI, immutability, resolvability,
  completeness
  preconditions:
    - "HasDOI(article_version)" \u2227 CrossrefRegistered(doi)"
    - "IsPreserved(article_version)"
    - "HasValidJATS(article_version)"
  postconditions:
    - "\u00acMutates(article_version)" # leaving Published = creating a new version
```

version:	1.0.0
amended:	2026-08-09
date:	2026-08-09
sponsor:	CSA
rationale:	"Initial encoding of Vol I Ch 11 \u00a711.5 as MRC."
version:	1.0.0
status:	active

4.9 State Machine Transitions (class: transition)

The publication state machine has 17 transitions. Each transition specifies the source state, the target state, the trigger, the side effects, and whether the transition is idempotent.

Attribute	Value
Class enum value	transition
Provision count	17
Source chapter	Vol I Ch 11
Class-specific fields	from, to, trigger, side_effects, idempotent
Schema file	schemas/transition.schema.json

4.9.1 Class-Specific Fields

Field	Type	Required	Description
from	string	REQUIRED	Source state (STATE-*).
to	string	REQUIRED	Target state (STATE-*).
trigger	string	REQUIRED	Triggering event or command (e.g., DOI_MINTED, editor-approve).
side_effects	array	REQUIRED	List of side effects (events emitted, external calls, notifications sent).
idempotent	boolean	REQUIRED	Whether the transition is idempotent (re-applying it has no additional effect).
predicate	string	REQUIRED (universal)	Guard predicate: the formal condition under which the transition is permitted.

4.9.2 Example: DOI Minted → Published

```
# constitution/state-machine/transitions.yaml (excerpt)
-
  id: TRANS-DOI-Minted-to-Published
  class: transition
  statement: "Transition from DOI-Minted to Published after preservation deposit
  completes."
  source_chapter: "Vol I Ch 11 \u00a711.6"
  severity: SEV-1
  owner: Platform Engineering
  predicate: "State(v) = DOI-Minted \u2227 IsPreserved(v) \u2192 State(v) := Published"
  verification_method: >-
  Integration test: drive an article version through DOI-Minted;
  complete preservation; verify state transitions to Published
  within 60s; verify EVT-22 is emitted.
  required_evidence:
  type: transition_test_report
  format: json
  max_age: 24h
  storage: certification-repository/transitions/TRANS-DOI-Minted-to-Published/
  verifier: automated
  automation:
  policy_module: policies/TRANS-DOI-Minted-to-Published.rego
  ci_job: .github/workflows/verify-state-machine.yml [TRANS-DOI-Minted-to-Published]
  monitor: TRANS-MON-DOI-Minted-to-Published
  alert: TRANS-DOI-Minted-to-Published-violation
  slo: null
  certification_rule: CC-TRANS-DOI-Minted-to-Published
  traceability:
  architecture: [ARCH-publication-service, ARCH-preservation-service]
  adr: [ADR-002-service-decomposition, ADR-019-preservation-pipeline]
  database: [article_versions.state]
  api: []
  code: [services/publication/state_machine.py]
  storage: []
  test: [TEST-TRANS-DOI-Minted-to-Published-happy, TEST-TRANS-DOI-Minted-to-
  Published-idempotent]
  evidence: [EVID-2026-TRANS-DOI-Minted-to-Published-latest]
  from: STATE-DOI-Minted
  to: STATE-Published
  trigger: PRESERVATION_COMPLETED
  side_effects:
  - emit EVT-22-PublicationCompleted
  - notify submitting author
  - update search index
  idempotent: true
  version: 1.0.0
  amended:
  - date: 2026-08-09
```

rationale: "Initial encoding of Vol I Ch 11	sponsor: CSA
	\u00a711.6 as MRC."
status: active	version: 1.0.0

4.10 Certification Criteria (class: criterion)

Certification criteria are the binary pass/fail tests that gate every release. There are over 600 criteria, one per provision (some provisions have multiple criteria). Every criterion specifies its pass condition, its fail condition, its verification method, its required evidence, its maximum evidence age, its verifier, and its gating classification (GATE or INFO).

Attribute	Value
Class enum value	criterion
Provision count	600+
Source chapter	Vol I Ch 8 + Vol II Ch 22
Class-specific fields	pass_criterion, fail_criterion, verification_method, required_evidence, max_age, verifier, gating
Schema file	schemas/criterion.schema.json

4.10.1 Class-Specific Fields

Field	Type	Required	Description
pass_criterion	string	REQUIRED	The objective condition that MUST be true to PASS. SHALL be binary and testable.
fail_criterion	string	REQUIRED	The condition that constitutes FAIL. SHALL be the negation of PASS, plus evidence-missing conditions.
verification_method	string	REQUIRED	The procedure to check the criterion. SHALL produce objective evidence. Overrides the universal verification_method.
required_evidence	object	REQUIRED	Evidence contract. Overrides the universal required_evidence.

max_age	string	REQUIRED	Maximum evidence age (e.g., 1h, 24h).
verifier	enum	REQUIRED	Verifier role. Overrides the universal verifier.
gating	enum	REQUIRED	GATE (FAIL blocks release) or INFO (FAIL is recorded).

4.10.2 Example: CC-INV-I-02

```
# constitution/certification/criteria/CC-INV-I-02.yaml
id: CC-INV-I-02
class: criterion
statement: "Criterion for INV-I-02: exactly one DOI per published article version."
source_chapter: "Vol II Ch 22 \u00a722.1 (certification framework)"
severity: SEV-1
owner: Release Certification Authority
predicate: "\u2200 v : Published(v) \u2192 COUNT(DISTINCT dois WHERE
article_version_id = v) = 1"
verification_method: >-
  Execute the hourly reconciliation scan; retrieve the scan report
  from the certification repository; verify the report shows
  COUNT = 1 for every published article version.
required_evidence:
  type: reconciliation_report
  format: csv
  max_age: 1h
  storage: certification-repository/invariants/INV-I-02/
verifier: rca
certification_rule: CC-INV-I-02 # self-reference
traceability:
  architecture: [ARCH-publication-service, ARCH-doi-minter]
  adr: [ADR-007-doi-minting-strategy]
  database: [dois, article_versions]
  api: []
  code: [services/doi-minter/, validators/inv_i_02_rego.py]
  storage: []
  test: [TEST-INV-I-02-uniqueness, TEST-INV-I-02-crossref-reconciliation]
  evidence: [EVID-2026-INV-I-02-latest]
pass_criterion: "COUNT(DISTINCT dois WHERE article_version_id = V) = 1 for every
published V; AND every DOI resolves via Crossref REST API."
fail_criterion: "COUNT \u2260 1 for any V; OR any DOI does not resolve; OR evidence not
retrievable; OR evidence stale."
max_age: 1h
gating: GATE
version: 1.0.0
amended:
```



```
-
  date: 2026-08-09
  sponsor: RCA
  rationale: "Initial encoding of Vol II Ch 22 criterion for INV-I-02."
  version: 1.0.0
status: active
```

Chapter 5 — Cross-Cutting Schemas

Four sub-schemas are referenced by the universal schema and by several class schemas: the traceability object, the automation object, the required_evidence object, and the amended array. These are documented in this chapter. Each sub-schema is defined as a separate JSON Schema file (referenced via \$ref from provision.schema.json) and as a CUE definition (referenced via the package name from constitution.cue).

5.1 The traceability Object

The traceability object is the bidirectional-link index between a provision and its architectural realization. Every provision **MUST** specify at least one non-empty traceability field; the Traceability Matrix generator consumes this object to populate the Neo4j property graph (per RC2 Roadmap Chapter 3).

Field	Type	Required	Description
architecture	array of strings	OPTIONAL	Architecture component identifiers (ARCH-*). Resolved against the architecture component catalogue.
adr	array of strings	OPTIONAL	Architecture Decision Record identifiers (ADR-NNN-slug). Resolved against the ADR repository. Each MUST match ^ADR-\d{3}-.
database	array of strings	OPTIONAL	Database tables (and optionally columns in dot notation). Resolved against the database schema snapshot.
api	array of strings	OPTIONAL	API endpoint paths (HTTP method + path). Resolved against the OpenAPI 3.1 specification.
code	array of strings	OPTIONAL	Code module paths (file or directory). Resolved against the implementation repository.
storage	array of strings	OPTIONAL	Storage artifact URIs (s3://, gcs://, nfs://, etc.). Resolved against the

			storage manifest.
test	array of strings	OPTIONAL	Test identifiers (TEST-*). Resolved against the test repository. Each MUST match ^TEST-.
evidence	array of strings	OPTIONAL	Evidence record identifiers (EVID-*). Resolved against the certification repository. Each MUST match ^EVID-.

At least one of architecture, adr, code, or test MUST be non-empty. This minimum is enforced by Stage 3 (semantic validation) because it cannot be expressed in JSON Schema without allOf branches that would make the schema unreadable.

5.2 The automation Object

The automation object specifies the executable artifacts that enforce the provision at runtime. It is required when severity = SEV-1 or verifier = automated. Every referenced artifact MUST exist (referential validation, Stage 4).

Field	Type	Required	Description
policy_module	string	REQUIRED when verifier = automated	Path to the OPA/Rego policy module. MUST match ^policies/.+\.rego\$ and MUST resolve to an existing file.
ci_job	string	OPTIONAL	CI job identifier. SHOULD reference a workflow under .github/workflows/.
monitor	string	OPTIONAL	Monitor identifier. SHOULD be unique within the MRC. References a Prometheus monitor or a scheduled job.
alert	string	OPTIONAL	Alert rule identifier. SHOULD be unique within the MRC. References a Prometheus alerting rule or a PagerDuty rule.

5.3 The required_evidence Object

The required_evidence object specifies the artifact that demonstrates a provision's PASS. Evidence SHALL be objective, stored in the certification repository, retrievable by stable reference, within the maximum evidence age, sufficient to demonstrate the PASS condition, immutable once recorded, and attributable to a named verifier.

Field	Type	Required	Description
type	string	REQUIRED	Evidence type. Examples: reconciliation_report, hash_reconciliation_report, chain_walk_report, integration_test_report, latency_percentile_report, event_replay_report, slo_evaluation_report.
format	enum	REQUIRED	Format: csv, json, xml, pdf, png, log.
max_age	string	REQUIRED	Maximum age before evidence is treated as stale. MUST match <code>^\d+(s m h d w)\$</code> .
storage	string	REQUIRED	Storage path within the certification repository. MUST begin with certification-repository/.

5.4 The amended Array

The amended array is the chronological history of the provision's amendments. Every amendment — from the initial commit through every subsequent change — MUST be recorded as an entry in this array. The array SHALL contain at least one entry (the initial commit). Versions across entries MUST be strictly monotonic increasing.

Field	Type	Required	Description
date	string	REQUIRED	ISO 8601 date (YYYY-MM-DD) of the amendment commit.
sponsor	string	REQUIRED	The role that authorized the amendment (e.g., CSA, CISO, Platform Engineering,

			RCA).
rationale	string	REQUIRED	One-sentence rationale for the amendment. MUST be at least 8 characters. Cited in the human-readable amendment log.
version	string	REQUIRED	SemVer of the provision after this amendment. MUST be greater than the previous entry's version (or 1.0.0 for the initial commit).

Chapter 6 — Validation Rules

Validation is the process by which the MRC's structural, semantic, referential, and version-compatibility invariants are enforced. Validation runs in five stages on every commit (Chapter 2 §2.4). This chapter documents the rule sets for each stage. Every rule is normative: a YAML file that violates a rule **SHALL** fail validation and **SHALL** be rejected by the CI pipeline.

6.1 Structural Validation

Structural validation enforces the JSON Schema 2020-12 definitions: required fields, types, enums, patterns, `additionalProperties: false`, and the if/then/else branches expressed in the universal schema (§3.3) and the class schemas. The structural rules are:

Rule ID	Rule	Stage
S-1	Every provision MUST conform to its class schema.	2
S-2	<code>additionalProperties</code> is false on every schema; unknown fields SHALL fail.	2
S-3	Required fields MUST be present and non-null.	2
S-4	Enum fields MUST be one of the declared values.	2
S-5	Pattern fields MUST match their regex.	2
S-6	If <code>class = invariant</code> , <code>predicate</code> and <code>runtime_monitor</code> MUST be present (<code>allOf</code> branch).	2
S-7	If <code>severity = SEV-1</code> , <code>automation</code> MUST be present (<code>allOf</code> branch).	2
S-8	If <code>verifier = automated</code> , <code>automation</code> MUST be present (<code>allOf</code> branch).	2
S-9	If <code>status = deprecated</code> , <code>successor</code> and <code>sunset_date</code> MUST be present (<code>CUE</code> conditional).	2
S-10	<code>meta.yaml</code> MUST conform to <code>meta.schema.json</code> .	2

6.2 Semantic Validation

Semantic validation enforces cross-field constraints that cannot be expressed in JSON Schema without making the schema unreadable. The semantic rules are encoded in `validators/validate_mrc.py` and are documented below.

Rule ID	Rule	Stage
SEM-1	If severity = SEV-1, automation.policy_module MUST be set and MUST reference an existing file.	3
SEM-2	If verifier = automated, automation MUST be set.	3
SEM-3	runtime_monitor.cadence MUST be no slower than required_evidence.max_age.	3
SEM-4	At least one of traceability.architecture, traceability.adr, traceability.code, traceability.test MUST be non-empty.	3
SEM-5	verifier MUST NOT be the same team as owner of any code module cited in traceability.code (separation of duties).	3
SEM-6	If class = invariant, slo MUST be null.	3
SEM-7	If class = invariant, severity MUST be SEV-1.	3
SEM-8	If status = retired, no active provision MAY reference this provision via traceability.	3
SEM-9	certification_rule MUST reference an existing criterion file.	3
SEM-10	If class = slo, the slo field MUST be null (the provision IS the SLO; it cannot reference itself).	3
SEM-11	If class = state, the invariants array MUST list only existing INV-* provisions.	3
SEM-12	If class = transition, from and to MUST reference existing STATE-*	3

	provisions.	
--	-------------	--

6.3 Referential Validation

Referential validation enforces that every cross-reference in the MRC resolves to an existing artifact in the corresponding source of truth. Referential validation is what makes the MRC a closed system: nothing dangles.

Rule ID	Reference Type	Source of Truth	Stage
REF-1	traceability.adr (ADR-NNN-slug)	adr/ repository	4
REF-2	traceability.test (TEST-*)	test/ repository	4
REF-3	traceability.evidence (EVID-*)	certification-repository/	4
REF-4	traceability.code (paths)	implementation repository	4
REF-5	traceability.api (HTTP method + path)	OpenAPI 3.1 specification	4
REF-6	traceability.database (tables, columns)	database schema snapshot	4
REF-7	traceability.storage (URIs)	storage manifest	4
REF-8	automation.policy_module (policies/*.rego)	constitution/policies/	4
REF-9	automation.ci_job (workflow references)	.github/workflows/	4
REF-10	certification_rule (CC-*)	constitution/certification/criteria/	4
REF-11	slo (SLO-NN)	constitution/slos/	4
REF-12	state-machine invariants, from, to (INV-*, STATE-*)	constitution/invariants/, state-machine/	4

6.4 Version Compatibility Validation

Version compatibility validation enforces that every provision's version field is SemVer-compliant and that the amended array is monotonic. The rules are:

Rule ID	Rule	Stage
VER-1	version MUST match <code>^\d+\.\d+\.\d+\$</code>	5

	(SemVer 2.0.0).	
VER-2	amended MUST be non-empty and entries MUST be in chronological order (date ascending).	5
VER-3	Versions across amended entries MUST be strictly monotonic increasing.	5
VER-4	The most recent amended entry's version MUST equal the top-level version field.	5
VER-5	An amendment that changes severity, predicate, or class MUST be a MAJOR bump.	5
VER-6	An amendment that adds an optional field or loosens a constraint MUST be a MINOR bump.	5
VER-7	An amendment that changes only documentation, comments, or example values MUST be a PATCH bump.	5
VER-8	meta.yaml.schema_version bump MUST be accompanied by a corresponding update to every provision's version field if the schema change is MAJOR.	5
VER-9	Conventional Commits 1.0.0: every commit modifying the MRC MUST use a scope of constitution and a type of feat (MINOR), fix (PATCH), or refactor / breaking-change (MAJOR).	5

Chapter 7 — Worked Examples

This chapter presents six fully annotated provisions, one per major provision class. Each example is shown as a YAML code block followed by an annotation table that walks through every field, explaining the value chosen and the constraint that value satisfies. The examples are drawn from the actual MRC content; the YAML is the committed file content. Cross-references to the field reference (§3.2) and the class schema (§4.N) are given in the annotation column.

7.1 A Complete Invariant: INV-I-02

INV-I-02 (Exactly one DOI per published article version) is the canonical example of an invariant: SEV-1, formally predicated, runtime-monitored, policy-enforced, and certification-gated. The full YAML was presented in §4.1.2; the annotation is:

Field	Value	Annotation
id	INV-I-02	Identifier per §3.2.1; class prefix INV-I (identity).
class	invariant	Per §3.2.2; invariant class triggers predicate + runtime_monitor requirement (S-6).
statement	Exactly one DOI per published article version.	Single sentence, 51 chars; satisfies S-5 length bounds.
source_chapter	Vol II Ch 26 §26.1	Citation pattern per §3.2.4.
severity	SEV-1	Per SEM-7 (invariant $\Rightarrow$ SEV-1).
owner	Platform Engineering	Team per Playbook registry.
predicate	$\forall v : \text{Published}(v) \rightarrow \exists! d : \text{HasDOI}(v, d) \wedge \text{Registered}(d)$	First-order logic; $\exists!$ is unique existence.
verification_method	SQL: COUNT(DISTINCT dois ...) = 1 ...	Names method (SQL) and success criterion (= 1).
required_evidence	reconciliation_report / csv / 1h / certification-repository/invariants/INV-I-02/	Satisfies §5.3 sub-schema.
verifier	automated	Triggers SEM-2 (automation required).
automation	{policy_module: policies/INV-I-02.rego, ...}	Satisfies SEM-1 (SEV-1 $\Rightarrow$ automation) and SEM-2 (automated $\Rightarrow$ automation).
runtime_monitor	{id: INV-MON-I-02, cadence: hourly, ...}	Satisfies S-6 (invariant $\Rightarrow$ runtime_monitor); cadence satisfies SEM-3 ($\leq$ max_age 1h).
slo	null	Satisfies SEM-6 (invariant $\Rightarrow$ slo null).

certification_rule	CC-INV-I-02	Self-referential criterion per SEM-9.
traceability	{adr: [ADR-007], code: [services/doi-minter/], ...}	Satisfies SEM-4 (at least one of arch/adr/code/test non-empty); all refs resolve (REF-1, REF-4).
version	1.0.0	Satisfies VER-1 (SemVer).
amended	[[date: 2026-08-09, version: 1.0.0]]	Satisfies VER-2, VER-3, VER-4 (monotonic, single entry, top-level match).
status	active	Per §3.2.18.

7.2 A Complete Rule: PUB-001

PUB-001 (Every published article SHALL have exactly one DOI registered with Crossref) is a rule: SEV-1, runtime-enforced, Crossref-reconciled. The full YAML was presented in §4.2.3. Key annotations:

Field	Value	Annotation
id	PUB-001	Identifier per §3.2.1; rule prefix PUB.
class	rule	Per §3.2.2.
predicate	$\Diamond a : \text{Published}(a) \rightarrow \exists! d : \text{HasDOI}(a, d) \wedge \text{CrossrefRegistered}(d)$	Modal operator $\Diamond$ (always) for rule.
enforcement_layer	runtime	Per §4.2 class schema; runtime = request path.
enforcement_mechanism	publication-guard.py: assert has_single_doi(article) before state transition	Names the executable mechanism.
detection	{detection_point: runtime, detection_latency: 0s, alert_routing: SEV-1-page}	Zero-latency detection with SEV-1 paging.
slo	SLO-01	Cross-references availability SLO; satisfies REF-11.
version	1.0.1	MINOR bump (1.0.0 $\rightarrow$ 1.0.1) for the Crossref migration amendment; satisfies VER-6.
amended	[[version: 1.0.0], {version: 1.0.1}]	Two entries; monotonic (VER-3); top-level matches latest (VER-4).

7.3 A Complete Contract: SUB-10 DOI Minter

SUB-10 (DOI Minter) is the most complex contract in the MRC: nine class-specific fields, each an object or array. The full YAML was presented in §4.3.2. Key annotations:

Field	Value	Annotation
id	SUB-10	Per §3.2.1; contract prefix SUB.
class	contract	Per §3.2.2.
inputs	[[name: article_version_id, type: UUID, source: PublicationService, idempotency: ...]]	Per §4.3 class schema; idempotency contract present.
outputs	[[name: doi, type: string, destination: PublicationService, guarantee: exactly-once]]	Exactly-once guarantee is the strongest; required for SEV-1.
authority	{databases_write: [dois], external_services_call: [Crossref REST API], secrets_access: [crossref-api-credentials]}	Least-privilege authority.
failure_conditions	Crossref 5xx → retry; 4xx → no retry, alert; DB write fail after registration → compensating transaction	Three enumerated conditions with distinct responses.
dependencies	[PublicationService (critical), Crossref REST API (critical)]	Both critical: failure of either blocks publication.
performance_budget	{p50: 2s, p95: 5s, p99: 10s, throughput: 10/s, error_rate: < 0.1%}	Concrete budgets cross-referenced to LAT-002 and THP-005.
certification	{criteria: [CC-SUB-10], gating_severity: SEV-1}	GATE criterion.
acceptance_tests	[TEST-SUB-10-mint-and-register, TEST-SUB-10-idempotency, TEST-SUB-10-retry-on-crossref-5xx, TEST-SUB-10-compensating-transaction]	Four tests; each resolves via REF-2.
runtime_evidence	[structured_log, prometheus_metric, opentelemetry_trace]	Three signal types; each with storage location.

7.4 A Complete Event: EVT-22 PublicationCompleted

EVT-22 (PublicationCompleted) is the canonical end-of-publication event: four consumers, per-key ordering, transactional outbox. The full YAML was presented in §4.5.2. Key annotations:

Field	Value	Annotation
id	EVT-22	Per §3.2.1; event prefix EVT.

class	event	Per §3.2.2.
producer	SUB-01-submission	Single producer per §4.5.
consumer	[SUB-04-search, SUB-12-preservation, SUB-15-export, SUB-17-analytics]	Four consumers; each MUST exist (REF-12).
payload_schema	{type: object, required: [event_id, article_version_id, doi, published_at], ...}	Embedded JSON Schema; valid 2020-12 fragment.
ordering	per-key-ordering	Ordered by article_version_id; weakest sufficient guarantee.
idempotency_key	event_id	UUID event_id provides natural idempotency.
outbox_table	event_outbox	Transactional outbox per ADR-018.

7.5 A Complete SLO: SLO-01 Availability

SLO-01 (Availability) is the platform's headline reliability target: 99.9% over 28 days, four burn-rate alerts. The full YAML was presented in §4.6.2. Key annotations:

Field	Value	Annotation
id	SLO-01	Per §3.2.1; SLO prefix SLO.
class	slo	Per §3.2.2.
slo	null	Satisfies SEM-10 (slo provision IS the SLO).
sli_formula	sum(rate(http_requests_total{status=~"2.."}[28d])) / sum(rate(http_requests_total[28d]))	PromQL formula; valid syntax.
target	0.999	99.9% availability.
window	28d	Standard SLO window.
error_budget	0.001	1 - target.
burn_rate_alerts	4 alerts: 14.4×/1h (page SEV-1), 6.0×/6h (page SEV-2), 3.0×/24h (ticket SEV-2), 1.0×/24h (freeze SEV-3)	Google SRE multi-window multi-burn-rate pattern.

7.6 A Complete Criterion: CC-INV-I-02

CC-INV-I-02 is the certification criterion that gates INV-I-02 at release time: binary PASS/FAIL, GATE-class, RCA-verified. The full YAML was presented in §4.10.2. Key annotations:

Field	Value	Annotation
id	CC-INV-I-02	Per §3.2.1; criterion prefix CC-.
class	criterion	Per §3.2.2.
pass_criterion	COUNT(DISTINCT dois WHERE article_version_id = V) = 1 for every published V; AND every DOI resolves via Crossref REST API.	Binary, testable, complete.
fail_criterion	COUNT ≠ 1 for any V; OR any DOI does not resolve; OR evidence not retrievable; OR evidence stale.	Negation of PASS plus evidence-missing conditions.
max_age	1h	Matches INV-I-02's required_evidence.max_age.
verifier	rca	Release Certification Authority; satisfies separation of duties (SEM-5) because RCA does not own services/doi-minter/.
gating	GATE	FAIL blocks release.

Chapter 8 — Authoring Workflow

This chapter documents the mechanical workflow for changing the MRC. The workflow is procedural: it tells the engineer what to do, step by step, in what order, with what artifacts. The governance process that authorizes a change (Constitution amendment per Vol II Ch 36) is documented elsewhere; this chapter assumes the change is authorized and concerns itself with the mechanical execution.

8.1 Adding a New Provision

To add a new provision to the MRC, perform the following steps in order. Each step references the validation rule it satisfies.

6. **Step 1 — Reserve the identifier.** the next available identifier in the provision's category (e.g., INV-I-08 for the next identity invariant). Confirm the identifier is unused by grepping the repository.
7. **Step 2 — Create the YAML file.** in the correct directory (e.g., constitution/invariants/identity/INV-I-08.yaml). The file name MUST equal the lowercase id with conventional hyphenation (per §2.2).
8. **Step 3 — Populate the universal fields.** by copying the nearest existing provision of the same class and editing every field. Use the field reference (§3.2) and the class schema (§4.N) to determine required fields and valid values.
9. **Step 4 — Populate the class-specific fields.** (violation_response for invariants, enforcement_layer for rules, inputs/outputs/etc. for contracts, etc.) per the class schema.
10. **Step 5 — Populate the amended array.** with at least the initial-commit entry: {date: today, sponsor: <your team>, rationale: <one sentence>, version: 1.0.0}.
11. **Step 6 — Set version to 1.0.0 and status to active.** with value 1.0.0 and status field with value active.
12. **Step 7 — Validate locally.** validate the file against its class schema:

```
python validators/validate_mrc.py --stage structural --file path/to/new.yaml
```

 All five stages (structural, semantic, referential, version) SHOULD pass locally before commit.
13. **Step 8 — Write the OPA/Rego policy module.** for the new provision. If severity = SEV-1 or verifier = automated, the policy module is REQUIRED (SEM-1, SEM-2). The policy module MUST implement the predicate as an OPA/Rego rule.
14. **Step 9 — Write the acceptance test.** that exercises the provision. The test MUST be named TEST-{provision-id}-{scenario} and MUST be referenced in traceability.test.
15. **Step 10 — Commit with a Conventional.** Constitutional Commit message format:
 feat(constitution): add INV-I-08 (one ISSN per journal)

Encodes Vol II Ch 26 §26.1 (identity invariants) as
MRC. Adds YAML, Rego policy, acceptance test.

Per Conventional Commits 1.0.0 (VER-9).

16. **Step 11 — Push and open a pull request.** . The CI pipeline runs all five validation stages. If any stage fails, fix the file and re-push. If all stages pass, the generate-derived job regenerates every affected derived artifact and commits the result to constitution-built/.
17. **Step 12 — Reviewer reviews both.** the YAML change and the resulting prose change (in constitution-built/). The reviewer MUST verify: (a) the YAML conforms to the schema; (b) the predicate is expressible as Rego; (c) the traceability references resolve; (d) the generated prose is correct.
18. **Step 13 — Merge.** the pull request. The MRC now contains the new provision; derived artifacts are updated; the OPA policy bundle is published; the Backstage catalog is refreshed.

8.2 Amending an Existing Provision

To amend an existing provision, perform the following steps. Amendment is the most common operation; the workflow is therefore streamlined.

19. **Step 1 — Modify the field(s).** in the YAML file. Do not change the id, class, or source_chapter (changing any of these requires retirement and re-creation, not amendment).
20. **Step 2 — Bump the version.** according to the amendment class: PATCH for editorial (comments, examples); MINOR for additive (new optional field, loosened constraint); MAJOR for breaking (predicate change, severity change, removed field). See VER-5, VER-6, VER-7.
21. **Step 3 — Append to the amended array.** with the new entry: {date: today, sponsor: <authorizing role>, rationale: <one sentence>, version: <new version>}.
22. **Step 4 — Validate locally.** (stages 1-5). Pay special attention to VER-3 (monotonicity), VER-4 (top-level match), and any referential rules affected by the change.
23. **Step 5 — Update the Rego policy.** if the predicate changed, the Rego policy MUST be updated to match. Otherwise, the policy is unchanged.
24. **Step 6 — Commit with.** Constitutional Commit message format:
fix(constitution): amend PUB-001 to use Crossref REST API

Migrates from Event Data (sunset 2026-04-23) per ADR-022.

25. **Step 7 — Push, open PR.** and review both the YAML change and the generated prose diff.

8.3 Deprecating a Provision

To deprecate a provision (mark it as superseded by a successor), perform the following steps. Deprecation is reversible: the provision remains in the repository and can be re-activated if the successor is later withdrawn.

26. **Step 1 — Change.** the status field from active to deprecated. CUE will require the successor and sunset_date fields (S-9).

27. **Step 2 — Add successor.** with the id of the successor provision (e.g., INV-I-09 if INV-I-08 is being deprecated in favor of INV-I-09).
28. **Step 3 — Add sunset_date.** the calendar date by which the deprecated provision will be retired. Typically 90 days from the deprecation commit.
29. **Step 4 — Append to amended.** with the new entry: {date: today, sponsor: CSA, rationale: <one sentence naming the successor>, version: <MAJOR bump>}.
30. **Step 5 — Bump version (MAJOR).** Per VER-5, deprecation is a MAJOR bump.
31. **Step 6 — Add a back-reference.** in traceability (in the successor's traceability.replaces array). The successor provision SHOULD include a replaces field pointing back.
32. **Step 7 — Validate locally.** (stages 1-5). Stage 3 verifies SEM-8 (no active provision references a retired provision); for deprecated, the rule is relaxed (active provisions MAY still reference deprecated ones, with a warning).
33. **Step 8 — Commit.** Constitutional Commit message:
refactor(constitution)!: deprecate INV-I-08 in favor of INV-I-09

BREAKING CHANGE: INV-I-08 is deprecated; sunset 2026-12-01.

8.4 Removing a Provision

Removing a provision from the MRC is not a mechanical operation. It is a Constitution amendment per Vol II Chapter 36 and requires the full governance process (CSA sponsorship, Engineering Leadership Team approval, 14-day review window). After governance approval, the mechanical steps are:

34. **Step 1 — Confirm.** the provision has been deprecated (status = deprecated) for at least 90 days and that the sunset_date has passed.
35. **Step 2 — Confirm.** no active provision references the to-be-retired provision via traceability (SEM-8). If any active provision references it, those references MUST be removed first (by amending the referencing provisions).
36. **Step 3 — Change status.** from active to retired. The provision MUST remain in the repository; the YAML file is NOT deleted. Retired provisions are excluded from the generated Constitution volumes but remain in the repository for audit history.
37. **Step 4 — Append to amended.** with the new entry: {date: today, sponsor: CSA, rationale: <governance decision reference>, version: <MAJOR bump>}.
38. **Step 5 — Retire.** the Rego policy file (policies/{id}.rego) is moved to policies/retired/ rather than deleted, preserving the audit trail. The CI job that loaded the policy is updated to skip retired policies.
39. **Step 6 — Commit.** Constitutional Commit message:
refactor(constitution)!: retire INV-I-08 per governance decision GOV-2026-014

Why retire, not delete?

Deleting a YAML file would break the audit trail: every historical Constitution volume, every historical certification report, and every historical traceability graph snapshot references provisions by id. If the file were deleted, those references would dangle. Retirement preserves the file (and therefore the references) while excluding the provision from current derived artifacts. The storage cost is negligible; the audit-trail cost of deletion is not.

Chapter 9 — Counterarguments and Limitations

The Machine-Readable Constitution is an architectural choice, not a law of nature. This chapter documents the limitations of the MRC approach and answers the most common counterarguments raised against it. The aim is not advocacy but honesty: a reference that documents only the strengths of its subject is not a reference, it is marketing.

9.1 Limitation: YAML and JSON Schema Cannot Express Every Constraint

YAML is a serialization format, not a constraint language. JSON Schema 2020-12 is a constraint language, but a limited one: it cannot express cross-field dependencies with the clarity of a real programming language, it cannot express monotonic-list constraints (e.g., VER-3: amended versions **MUST** be strictly monotonic), and it cannot express referential integrity (REF-1 through REF-12). The MRC addresses this in three ways: (a) the CUE source (§3.4) expresses cross-field constraints natively and compiles to JSON Schema; (b) the semantic and referential rules are encoded in Python (validators/validate_mrc.py) and executed in CI; (c) the deeper constraints (e.g., the Rego policy **MUST** implement the predicate) are encoded in Rego and evaluated against the runtime evidence. No single tool suffices; the MRC uses three (CUE, Python, Rego) and accepts the complexity.

9.2 Limitation: Schema Evolution Creates Migration Burden

Every MAJOR bump of meta.yaml.schema_version requires every existing provision to be migrated. The MRC currently encodes 558 provisions ($38 + 47 + 20 + 276 + 34 + 10 + 103 + 13 + 17 + 600 = 1,158$ files, including 600+ criteria); a MAJOR schema change today would require editing over a thousand files. The MRC addresses this by (a) preferring MINOR over MAJOR changes wherever possible (additive fields, optional defaults), (b) writing migration scripts in the same commit as the schema change (the migration script is a Python script that reads every YAML file, applies the transformation, and writes the file back), and (c) requiring CSA sponsorship for every MAJOR change (a governance friction that makes MAJOR changes rare). The burden is real and is the price of a closed, validated schema.

9.3 Limitation: Authors May Encode Provisions That Pass Schema but Violate Intent

A schema can validate that a predicate field is a non-empty string; it cannot validate that the predicate is the right predicate for the provision's intent. An author who writes INV-I-02's predicate as " $\forall v : \text{True}$ " (vacuously true) would pass structural, semantic, and referential validation while encoding an invariant that means nothing. The MRC addresses this in two ways: (a) the Rego policy **MUST** implement the predicate, and the policy is reviewed by a human reviewer in the pull request; (b) the acceptance test **MUST** exercise the predicate, and the test is reviewed in the same pull request. The defense is procedural (human review of predicate, policy, and test together), not structural. This is a fundamental

limitation of any machine-readable specification: the specification can encode the letter of the law but not its spirit.

9.4 Counterargument: “Why Not Just Use OpenAPI?”

OpenAPI 3.1 is a mature, widely-supported specification format for HTTP APIs. It is the natural choice for the MRC's contract class (§4.3) and is referenced from the `traceability.api` field. But OpenAPI is API-centric: it describes HTTP endpoints, request/response schemas, and authentication. It does not describe invariants, rules, events, SLOs, thresholds, state machines, or certification criteria — the other nine provision classes of the MRC. Stretching OpenAPI to cover these classes (via x- extensions, vendor-specific keywords, or wrapper documents) would produce a specification that is neither idiomatic OpenAPI nor a clean schema. The MRC uses JSON Schema 2020-12 (which OpenAPI 3.1 is built on) as the common substrate and uses OpenAPI 3.1 itself only for the narrow purpose it was designed for: HTTP API documentation. The two coexist; neither subsumes the other.

9.5 Counterargument: “Why Not Just Use Prose?”

The RC1 Constitution was prose. Three hundred and three pages of Volume I, two hundred and sixty-nine pages of Volume II, audited quarterly by the RCA. The prose Constitution worked, in the sense that it governed the platform and was enforced (manually) at every release. So why convert to a machine-readable form? Four reasons. First, manual enforcement does not scale: 558 provisions, each verified by hand at every release, is a multi-day exercise that consumes the RCA's entire bandwidth. Second, manual enforcement is error-prone: the RCA occasionally misses provisions, especially after amendments. Third, prose cannot be queried: there is no way to ask “which provisions depend on ADR-007?” without reading every page. Fourth, prose cannot be diffed: a constitutional amendment produces a 300-page revised document with no machine-readable change set. The MRC addresses all four: automated enforcement (CI), automated verification (Rego policies), graph queries (Neo4j), and structural diffs (yq). The cost is the authoring burden documented in this reference; the benefit is a Constitution that scales with the platform.

Chapter 10 — Conclusion and Implications

10.1 Summary

This reference has documented the constitution.yaml schema in full. Chapter 1 introduced the MRC as the data form of the Constitution. Chapter 2 documented the repository structure: the directory tree, the file naming conventions, the meta.yaml entry point, and the five-stage validation pipeline. Chapter 3 documented the universal eighteen-field schema, field by field, with the JSON Schema 2020-12 definition and the CUE source. Chapter 4 documented the ten provision-class schemas, with two or three worked examples each. Chapter 5 documented the four cross-cutting sub-schemas (traceability, automation, required_evidence, amended). Chapter 6 documented the validation rule sets for each stage. Chapter 7 presented six fully annotated examples. Chapter 8 documented the authoring workflow for adding, amending, deprecating, and retiring provisions. Chapter 9 acknowledged the limitations and answered the common counterarguments.

The MRC is not a small system. It is over a thousand files, governed by a closed schema, validated by five CI stages, consumed by eight document generators, queried by a Neo4j graph, enforced by an OPA policy bundle, and monitored by a Grafana dashboard. The complexity is real. The justification is also real: a Constitution that cannot be queried, diffed, or automatically enforced cannot scale with the platform it governs. The MRC is the price of a Constitution that scales.

10.2 Implications for Tooling

Every tool in the RC2 architecture consumes the MRC. The implications for tooling engineers are:

- **Schema, not prose.** Every tool MUST consume the schema, not the prose. Tools that consume the prose (e.g., a script that greps the Constitution PDF) are fragile; tools that consume the schema are robust. The schema is the contract.
- **Tolerate additive change.** Every tool MUST tolerate additive schema changes (MINOR bumps) without code changes. A tool that breaks on a new optional field is a bug in the tool, not in the schema. MAJOR bumps require coordinated migration and are announced in meta.yaml.schema_version.
- **Validate before consume.** Every tool MUST validate its input against the schema before consuming it. A tool that processes an invalid YAML file (e.g., one missing a required field) is itself buggy. Use jsonschema 4.21 (or equivalent) as the first operation.
- **Handle retired provisions.** Every tool MUST handle retired provisions gracefully. A retired provision is excluded from generated artifacts but remains in the repository; tools MUST skip retired provisions (status = retired) and MUST NOT crash on them.
- **Deterministic output.** Every tool MUST emit deterministic output. Given the same MRC commit, every tool MUST produce byte-identical output. Non-determinism (e.g., dict ordering, timestamp injection) breaks reproducible builds and constitutional diffing.

10.3 Future Evolution

The MRC is at schema_version 1.0.0. Two evolutionary directions are anticipated. First, CUE may replace YAML as the authoring form. CUE's type system, defaults, and compile-time evaluation make it a strictly more powerful authoring language than YAML; the cost is a smaller tooling ecosystem and a steeper learning curve. The current architecture uses CUE only for the schema source; a future architecture may use CUE for the provisions themselves, with YAML as the compiled output. Second, schema federation may allow the MRC to import provisions from external repositories (e.g., a shared library of security invariants maintained by the Security Engineering team). The current architecture is monolithic; a federated architecture would require \$ref across repositories, a registry of federated schemas, and a tighter versioning discipline. Neither direction is imminent; both are anticipated. This reference will be revised when either is adopted.

The Constitution as Data — Revisited

The MRC began as a proposal in the RC2 Evolution Roadmap: convert the Constitution from a document into a database. This reference is the specification of that database. The conversion is complete when every engineer, every tool, and every process treats the MRC as the Constitution and the prose volumes as generated views. That conversion is the work of RC2 and beyond. This reference is the contract that makes the conversion safe.

Appendices

Appendix A — Consolidated JSON Schema 2020-12 Definition

This appendix presents the consolidated JSON Schema 2020-12 definition that combines the universal schema (§3.3) with the class schemas (§4.N). The consolidated schema is the single file against which every provision in the MRC is validated. The class-specific schemas are expressed as oneOf branches; the universal fields are factored into the base definition and the class branches add only the class-specific fields.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://opuspublica.com/schemas/mrc/1.0.0/provision.consolidated.schema.json",
  "title": "Consolidated Provision Schema (all classes)",
  "type": "object",
  "additionalProperties": false,
  "required": ["id", "class", "statement", "source_chapter",
    "severity", "owner", "verification_method", "required_evidence",
    "verifier", "certification_rule", "traceability",
    "version", "amended", "status"],
  "properties": {
    "id": {"type": "string"},
    "class": {"enum": ["invariant", "rule", "contract",
      "field", "event", "slo", "threshold",
      "state", "transition", "criterion"]},
    "statement": {"type": "string"},
    "source_chapter": {"type": "string"},
    "severity": {"enum": ["SEV-1", "SEV-2", "SEV-3"]},
    "owner": {"type": "string"},
    "predicate": {"type": "string"},
    "verification_method": {"type": "string"},
    "required_evidence": {"$ref": "#/definitions/Evidence"},
    "verifier": {"enum": ["rca", "qa", "automated", "auditor"]},
    "automation": {"$ref": "#/definitions/Automation"},
    "runtime_monitor": {"$ref": "#/definitions/RuntimeMonitor"},
    "slo": {"type": ["string", "null"]},
    "certification_rule": {"type": "string"},
    "traceability": {"$ref": "#/definitions/Traceability"},
    "version": {"type": "string"},
    "amended": {"type": "array", "items": {"$ref": "#/definitions/Amendment"}},
    "status": {"enum": ["active", "deprecated", "retired"]},
    "successor": {"type": "string"},
    "sunset_date": {"type": "string"},
    "replaces": {"type": "array", "items": {"type": "string"}},
    "violation_response": {"$ref": "#/definitions/ViolationResponse"},
    "enforcement_layer": {"enum": ["ci", "deploy", "runtime", "audit"]},
```

```

        "enforcement_mechanism": {"type": "string"},
    "detection": {"$ref": "#/definitions/Detection"},
    "inputs": {"type": "array", "items": {"$ref": "#/definitions/Input"}},
    "outputs": {"type": "array", "items": {"$ref": "#/definitions/Output"}},
    "authority": {"$ref": "#/definitions/Authority"},
    "failure_conditions": {"type": "array", "items": {"type": "object"}},
    "dependencies": {"type": "array", "items": {"type": "object"}},
    "performance_budget": {"type": "object"},
    "certification": {"type": "object"},
    "acceptance_tests": {"type": "array", "items": {"type": "string"}},
    "runtime_evidence": {"type": "array", "items": {"type": "object"}},
    "db_table": {"type": "string"},
    "db_column": {"type": "string"},
    "storage_path": {"type": ["string", "null"]},
    "producer": {"type": "string"},
    "consumer": {"type": "array", "items": {"type": "string"}},
    "validation": {"type": "object"},
    "mutability": {"enum": ["immutable-on-publish",
                           "mutable-pre-publish",
                           "append-only", "mutable"]},
    "migration": {"type": "object"},
    "payload_schema": {"type": "object"},
    "ordering": {"enum": ["per-key-ordering",
                          "global-ordering", "unordered"]},
    "idempotency_key": {"type": "string"},
    "outbox_table": {"type": "string"},
    "sli_formula": {"type": "string"},
    "target": {"type": "number"},
    "window": {"enum": ["5m", "1h", "6h",
                        "24h", "7d", "28d", "90d"]},
    "error_budget": {"type": "number"},
    "burn_rate_alerts": {"type": "array", "items": {"type": "object"}},
    "budget_class": {"enum": ["latency", "throughput",
                              "batch", "queue", "storage"]},
    "operation": {"type": "string"},
    "p50": {"type": "string"},
    "p95": {"type": "string"},
    "p99": {"type": "string"},
    "hard_limit": {"type": "string"},
    "soft_warning": {"type": "string"},
    "invariants": {"type": "array", "items": {"type": "string"}},
    "preconditions": {"type": "array", "items": {"type": "string"}},
    "postconditions": {"type": "array", "items": {"type": "string"}},
    "from": {"type": "string"},
    "to": {"type": "string"},
    "trigger": {"type": "string"},
    "side_effects": {"type": "array", "items": {"type": "string"}},
    "idempotent": {"type": "boolean"},

```



```

      "pass_criterion": {"type": "string"},
      "fail_criterion": {"type": "string"},
      "max_age": {"type": "string"},
      "gating": {"enum": ["GATE", "INFO"]}
    },
    "oneOf": [
      {
        "properties": {"class": {"const": "invariant"}},
        "required": ["predicate", "runtime_monitor", "violation_response"],
        "properties": {"class": {"const": "rule"}},
        "required": ["predicate", "enforcement_layer", "enforcement_mechanism", "detection"],
        "properties": {"class": {"const": "contract"}},
        "required": ["inputs", "outputs", "authority", "failure_conditions", "dependencies", "performance_budget", "certification", "acceptance_tests", "runtime_evidence"],
        "properties": {"class": {"const": "field"}},
        "required": ["db_table", "db_column", "producer", "consumer", "validation", "mutability", "migration"],
        "properties": {"class": {"const": "event"}},
        "required": ["producer", "consumer", "payload_schema", "ordering", "idempotency_key", "outbox_table"],
        "properties": {"class": {"const": "slo"}},
        "required": ["sli_formula", "target", "window", "error_budget", "burn_rate_alerts"],
        "properties": {"class": {"const": "threshold"}},
        "required": ["budget_class", "operation", "p50", "p95", "p99", "hard_limit", "soft_warning"],
        "properties": {"class": {"const": "state"}},
        "required": ["invariants", "preconditions", "postconditions"],
        "properties": {"class": {"const": "transition"}},
        "required": ["from", "to", "trigger", "side_effects", "idempotent"],
        "properties": {"class": {"const": "criterion"}},
        "required": ["pass_criterion", "fail_criterion", "max_age", "gating"]
      },
      {
        "definitions": {
          "Evidence": {"type": "object", "required": ["type", "format", "max_age", "storage"]},
          "Automation": {"type": "object", "required": ["id", "cadence", "implementation"]},
          "Traceability": {"type": "object", "required": ["date", "sponsor", "rationale", "version"]},
          "ViolationResponse": {"type": "object"},
          "Detection": {"type": "object"}
        }
      }
    ]
  }

```

```

      "Input":
      "Output":
      "Authority":
    {"type": "object"},
    {"type": "object"},
    {"type": "object"}
  }

```

Appendix B — YAML Quick-Reference Card

This appendix is a one-page quick-reference card for the most common authoring operations. It is intended to be printed and kept beside the engineer's terminal.

Operation	Required Fields (beyond universal)	Example
Invariant	predicate, runtime_monitor, violation_response	INV-I-02: $\forall v: \text{Published}(v) \rightarrow \exists! d: \text{HasDOI}(v,d)$
Rule	predicate, enforcement_layer, enforcement_mechanism, detection	PUB-001: runtime, publication-guard.py
Contract	inputs, outputs, authority, failure_conditions, dependencies, performance_budget, certification, acceptance_tests, runtime_evidence	SUB-10: DOI Minter
Field	db_table, db_column, producer, consumer, validation, mutability, migration	article_doi: article_versions.doi, immutable-on-publish
Event	producer, consumer, payload_schema, ordering, idempotency_key, outbox_table	EVT-22: per-key-ordering, event_outbox
SLO	sli_formula, target, window, error_budget, burn_rate_alerts	SLO-01: 99.9% / 28d
Threshold	budget_class, operation, p50, p95, p99, hard_limit, soft_warning	LAT-001: 200ms p95 api-read
State	invariants, preconditions, postconditions	Published: [INV-I-02, INV-M-01, ...]
Transition	from, to, trigger, side_effects, idempotent	DOI-Minted $\rightarrow$ Published on PRESERVATION_COMPLETED
Criterion	pass_criterion, fail_criterion, max_age, gating	CC-INV-I-02: COUNT=1, GATE

Appendix C — Validator Installation and Usage

The MRC validator is a Python package named `mrc-validate`, hosted in the `constitution/` repository under `validators/`. This appendix documents installation and usage.

C.1 Installation

```
# Clone the MRC repository
git clone git@github.com:opuspublica/constitution.git
cd constitution

# Create a virtual environment (Python 3.12+)
python3.12 -m venv .venv
source .venv/bin/activate

# Install validator dependencies
pip install -r validators/requirements.txt
# jsonschema==4.21
# pyyaml==6.0
# requests==2.32 # for referential validation against external repos
# cue==0.9 # optional; for CUE → JSON Schema regeneration

# Install the validator as a package (editable)
pip install -e validators/
```

C.2 Usage

```
# Validate a single file (all stages)
python -m mrc_validate --file invariants/identity/INV-I-02.yaml

# Validate a single file at a specific stage
python -m mrc_validate --file path/to/file.yaml --stage structural
python -m mrc_validate --file path/to/file.yaml --stage semantic
python -m mrc_validate --file path/to/file.yaml --stage referential
python -m mrc_validate --file path/to/file.yaml --stage version

# Validate the entire repository (all files, all stages)
python -m mrc_validate --all

python -m mrc_validate --changed # uses git diff to identify files

# Regenerate schemas from CUE source (commit the result)
cue export cue/constitution.cue --out json --force
git diff --exit-code schemas/ # must be no-op

# Generate derived artifacts
python generators/gen_constitution_vol1.py
python generators/gen_constitution_vol2.py
python generators/gen_playbook.py
python generators/gen_traceability_matrix.py
python generators/gen_certification_checklist.py
python generators/gen_oscal.py
```

```
python generators/gen_opa_bundle.py
python generators/gen_backstage_catalog.py
```

C.3 Sample Output

```
$ python -m mrc_validate --file invariants/identity/INV-I-02.yaml
[Stage 1] CUE → JSON Schema ..... PASS (no schema changes)
[Stage 2] Structural validation ..... PASS
[Stage 3] Semantic validation ..... PASS
[Stage 4] Referential validation ..... PASS (8/8 refs resolved)
[Stage 5] Version validation ..... PASS (1.0.0; 1 amendment)

Overall: PASS (5/5 stages)

$ python -m mrc_validate --file invariants/identity/INV-I-02-broken.yaml
[Stage 1] CUE → JSON Schema ..... PASS
[Stage 2] Structural validation ..... FAIL
- /severity: 'SEV-4' is not one of ['SEV-1','SEV-2','SEV-3']
- /automation: required when severity='SEV-1' (allOf branch)
[Stage 3] Semantic validation ..... SKIPPED (Stage 2 failed)
[Stage 4] Referential validation ..... SKIPPED
[Stage 5] Version validation ..... SKIPPED

Overall: FAIL (1/5 stages passed)
Exit code: 1
```

Appendix D — References

The following references are cited normatively or informatively in this document. URLs were verified at the issue date.

#	Reference	URL	Role
1	JSON Schema 2020-12	https://json-schema.org/draft/2020-12/schema	normative
2	YAML 1.2.2	https://yaml.org/spec/1.2.2/	normative
3	CUE	https://cuelang.org	normative (for §3.4)
4	OpenAPI 3.1.0	https://spec.openapis.org/oas/v3.1.0	informative
5	RFC 2119 — Key words for use in RFCs to Indicate Requirement Levels	https://rfc-editor.org/info/rfc2119	normative
6	Semantic Versioning 2.0.0	https://semver.org	normative
7	Conventional Commits 1.0.0	https://conventionalcommits.org/	normative (for §8)

		en/v1.0.0/	
8	RFC 6902 — JavaScript Object Notation (JSON) Patch	https://rfc-editor.org/info/rfc6902	informative
9	RFC 7386 — JSON Merge Patch	https://rfc-editor.org/info/rfc7386	informative
10	OPA / Rego	https://www.openpolicyagent.org	informative
11	Neo4j Property Graph Model	https://neo4j.com/docs/getting-started/property-graph/	informative
12	OpenTelemetry	https://opentelemetry.io	informative
13	Google SRE — Service Level Objectives	https://sre.google/workbook/service-level-objectives/	informative
14	NIST SP 800-137 — Information Security Continuous Monitoring	https://csrc.nist.gov/pubs/sp/800/137/final	informative
15	NIST SP 800-204C — Policy-as-Code	https://csrc.nist.gov/pubs/sp/800/204/c/final	informative
16	SLSA — Supply-chain Levels for Software Artifacts	https://slsa.dev	informative
17	Sigstore (cosign, rekor, fulcio)	https://sigstore.dev	informative
18	CycloneDX (ECMA-424)	https://cyclonedx.org	informative
19	SPDX (ISO/IEC 5962:2021)	https://spdx.dev	informative
20	in-toto Attestation Framework	https://in-toto.io/attestation/	informative
21	OSCAL — Open Security Controls Assessment Language	https://pages.nist.gov/OSCAL/	informative
22	Backstage Software Catalog	https://backstage.io/docs/features/software-catalog/	informative
23	Crossref REST API	https://api.crossref.org/works	informative
24	Crossref Event Data sunset notice	https://www.crossref.org/blog/strengthening-support-for-data-citations-and-saying-goodbye-to-event-data/	informative

25	ORCID Public and Member APIs	https://info.orcid.org/what-is-orcid/services/	informative
26	CLOCKSS	https://clockss.org	informative
27	LOCKSS	https://lockss.org	informative
28	DOAJ API	https://doaj.org/api/	informative
29	Opus Publica RC1 Engineering Constitution	(this repository)	normative
30	Opus Publica RC2 Evolution Roadmap v1.0	(this repository)	informative

End of constitution.yaml Schema Reference, Version 1.0.